70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · MT4

Docker va Kubernetes platformalaridan foydalanib sun'iy intellekt modelini joylashtirish (Deployment) va AI xizmatlarini boshqarish bo'yicha amaliy loyiha

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/MT4
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/MT4
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu mustaqil ishda siz bitta AI xizmatini (tumor-api — o'sma belgilarini tasniflovchi model) boshidan oxirigacha joylashtirish loyihasini bajarasiz: FastAPI xizmati va avtomatik testlar, qatlamli konfiguratsiya va sog'liq tekshiruvlari, Dockerfile va docker-compose generatsiyasi va validatsiyasi, dev/prod Kubernetes manifestlari (Deployment, Service, HPA, ConfigMap, Secret, Ingress, PDB, NetworkPolicy), rolling update va canary, HPA va Cluster Autoscaler yuklama simulyatsiyasi hamda SLO asosidagi operatsion hisobot. Sandbox'da Docker va Kubernetes yo'q: xizmat jarayon ichida TestClient bilan ishlaydi, fayllar matn sifatida yaratilib Python'da tekshiriladi, klaster xulq-atvori esa simulyatsiya qilinadi. Vaqt, narx va yuklama raqamlari taxminiy.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/MT4`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → MT4 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. AI xizmati: FastAPI, model reestri, API kalit, probe'lar va avtomatik testlar
2. 2-qadam. Konfiguratsiya qatlamlari, chuqur readiness, probe'lar va to'g'ri to'xtash
3. 3-qadam. Dockerfile va docker-compose: generatsiya, validatsiya va qatlam keshi
4. 4-qadam. Kubernetes manifestlari: generator, dev/prod overlay va 18 qoidali tekshiruv
5. 5-qadam. Versiyani chiqarish: RollingUpdate simulyatsiyasi va haqiqiy so'rovlar bilan canary
6. 6-qadam. HPA (autoscaling/v2) + Cluster Autoscaler + Erlang-C: yuklama simulyatsiyasi
7. 7-qadam. Operatsion hisobot: SLO, xato byudjeti, burn-rate ogohlantirishlari va Go/No-Go
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. AI xizmati: FastAPI, model reestri, API kalit, probe'lar va avtomatik testlar

MAQSAD

Loyiha — «tumor-api»: sklearn breast_cancer (30 belgi) bo'yicha o'sma turini tasniflovchi xizmat. Ikki model versiyasi (v1 — logistik regressiya, v2 — tasodifiy o'rmon) o'qitilib, SHA-256 xeshi bilan reestrga (/tmp/models) yoziladi. Xizmat `lifespan` ichida modelni butunlik tekshiruvi va isitish bilan yuklaydi; /startupz, /livez, /readyz alohida, /v1/predict va /v1/predict/batch API kalit (x-api-key) talab qiladi, har javobga x-request-id va x-model-version sarlavhalari qo'shiladi, /metrics Prometheus matn formatida. Oxirida CI'dagi kabi 11 ta avtomatik test TestClient bilan bajariladi. MODEL_VERSION, MAX_BATCH ni o'zgartirib ko'ring.

> **Kutilgan natija:** v1 aniqligi ~0,96, v2 ~0,94; 11/11 PASS; ketma-ket ~300-400 so'rov/s; /metrics da predictions_total 133 va model_info.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. AI xizmati: FastAPI, model reestri (v1/v2), API kalit, so'rov ID, batch, probe'lar
va avtomatik testlar
# Loyiha: "tumor-api" — ko'krak bezi o'smasi belgilarini (sklearn breast_cancer, 30 belgi)
tasniflovchi xizmat.
# Docker/K8s o'rniga xizmat jarayon ichida TestClient bilan sinaladi — konteynerda ham aynan
shu kod ishlaydi.
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, pickle, time, uuid
from contextlib import asynccontextmanager
from typing import List

import numpy as np
from fastapi import FastAPI, Header, HTTPException, Request
from fastapi.responses import PlainTextResponse
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field, field_validator
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# --- Model reestri: ikki versiya o'qitiladi va /tmp/models/<versiya>/model.pkl ga yoziladi
(real loyihada — NAS/S3) ---
X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
REESTR = "/tmp/models"
for ver, est in (("v1", make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))),
                 ("v2", RandomForestClassifier(n_estimators=60, random_state=0))):
    est.fit(Xtr, ytr); os.makedirs(f"{REESTR}/{ver}", exist_ok=True)
    blob = pickle.dumps(est); open(f"{REESTR}/{ver}/model.pkl", "wb").write(blob)
    meta = {"versiya": ver, "aniqlik": float(est.score(Xte, yte)), "sha256":
hashlib.sha256(blob).hexdigest()[:16]}
    json.dump(meta, open(f"{REESTR}/{ver}/meta.json", "w"))
    print(f"reestr: {ver} aniqlik={meta['aniqlik']:.4f} sha256={meta['sha256']}")
```

```
# --- Sozlamalar muhit o'zgaruvchilaridan (12 omil) ---
MODEL_VERSION = os.getenv("MODEL_VERSION", "v1")
API_KEY = os.getenv("API_KEY", "maxfiy-demo-kalit")      # real loyihada — K8s Secret
MAX_BATCH = int(os.getenv("MAX_BATCH", "32"))
S = {"model": None, "meta": None, "startup": False, "ready": False, "n": 0, "xato": 0, "lat":
[], "boshlangan": time.time()}


@asynccontextmanager
async def lifespan(app):
    yo = f"{REESTR}/{MODEL_VERSION}"
    S["meta"] = json.load(open(f"{yo}/meta.json"))
    blob = open(f"{yo}/model.pkl", "rb").read()
    assert hashlib.sha256(blob).hexdigest()[:16] == S["meta"]["sha256"], "model fayli
buzilgan!"   # butunlik tekshiruvi
    S["model"] = pickle.loads(blob)
    S["model"].predict_proba(np.zeros((1, 30)))           # isitish (warm-up)
    S["startup"] = S["ready"] = True
    yield
    S["ready"] = False                                    # SIGTERM: trafikni qabul qilishni
to'xtatish (drain)


app = FastAPI(title="tumor-api", version="1.0.0", lifespan=lifespan)


@app.middleware("http")
async def sorov_id(request: Request, call_next):
    rid = request.headers.get("x-request-id") or uuid.uuid4().hex[:12]
    t0 = time.perf_counter(); resp = await call_next(request)
    resp.headers["x-request-id"] = rid
    resp.headers["x-model-version"] = MODEL_VERSION
    if request.url.path.startswith("/v1/"):
        S["lat"].append(time.perf_counter() - t0)
    return resp


class Namuna(BaseModel):
    belgilar: List[float] = Field(..., description="30 ta son belgisi")

    @field_validator("belgilar")
    @classmethod
    def uzunlik(cls, v):
        if len(v) != 30: raise ValueError("aynan 30 ta belgi kerak")
        if not all(np.isfinite(v)): raise ValueError("NaN/inf mumkin emas")
        return v


class Paket(BaseModel):
    namunalar: List[Namuna]


def kalit(x_api_key):
    if x_api_key != API_KEY:
        S["xato"] += 1; raise HTTPException(401, "API kalit noto'g'ri")


@app.get("/startupz")
def startupz():
    return {"ok": S["startup"]}


@app.get("/livez")
```

```
def livez():
    return {"ok": True, "uptime_s": round(time.time() - S["boshlangan"], 1)}


@app.get("/readyz")
def readyz():
    if not S["ready"]: raise HTTPException(503, "tayyor emas")
    return {"ok": True, "model": S["meta"]["versiya"]}


@app.get("/version")
def version():
    return {"xizmat": app.version, **S["meta"]}


@app.post("/v1/predict")
def predict(n: Namuna, x_api_key: str = Header(None)):
    kalit(x_api_key)
    p = S["model"].predict_proba(np.array([n.belgilar]))[0]
    S["n"] += 1
    return {"sinf": "xavfsiz" if p[1] >= 0.5 else "xavfli", "ehtimol": round(float(max(p)), 4)}


@app.post("/v1/predict/batch")
def batch(pk: Paket, x_api_key: str = Header(None)):
    kalit(x_api_key)
    if len(pk.namunalar) > MAX_BATCH: raise HTTPException(413, f"paket <= {MAX_BATCH}")
    P = S["model"].predict_proba(np.array([n.belgilar for n in pk.namunalar]))
    S["n"] += len(P)
    return {"natija": [int(p[1] >= 0.5) for p in P]}


@app.get("/metrics", response_class=PlainTextResponse)
def metrics():
    lat = sorted(S["lat"]) or [0.0]
    return (f"# TYPE predictions_total counter\npredictions_total {S['n']}\n"
            f"# TYPE auth_errors_total counter\nauth_errors_total {S['xato']}\n"
            f"# TYPE request_latency_p95_seconds gauge\nrequest_latency_p95_seconds
{lat[int(0.95 * (len(lat) - 1))]:.6f}\n"
            f"model_info{{version=\"{S['meta']['versiya']}\",sha=\"{S['meta']['sha256']}\"}}
1\n")


# --- Avtomatik test to'plami (CI'da pytest qiladigan ish) ---
K = {"x-api-key": API_KEY}
natija = []
def test(nom, shart):
    natija.append((nom, "PASS" if shart else "FAIL"))

with TestClient(app) as c:
    test("startupz 200", c.get("/startupz").json()["ok"])
    test("readyz 200", c.get("/readyz").status_code == 200)
    r = c.post("/v1/predict", json={"belgilar": Xte[0].tolist()}, headers=K)
    test("predict 200 + sinf", r.status_code == 200 and r.json()["sinf"] in ("xavfli",
"xavfsiz"))
    test("x-request-id sarlavhasi", len(r.headers.get("x-request-id", "")) == 12)
    test("so'rov ID saqlanadi", c.get("/livez", headers={"x-request-id": "abc123"}).headers["x-
request-id"] == "abc123")
    test("API kalitsiz -> 401", c.post("/v1/predict", json={"belgilar":
Xte[0].tolist()}).status_code == 401)
    test("29 belgi -> 422", c.post("/v1/predict", json={"belgilar": [0.0] * 29},
headers=K).status_code == 422)
    test("batch 33 -> 413", c.post("/v1/predict/batch", json={"namunalar": [{"belgilar": [0.0]
```

```
* 30}] * 33}, headers=K).status_code == 413)
    rb = c.post("/v1/predict/batch", json={"namunalar": [{"belgilar": x.tolist()} for x in
Xte[:32]]}, headers=K)
    test("batch 32 aniqlik >= 0.9", rb.status_code == 200 and np.mean(np.array(rb.json()
["natija"]) == yte[:32]) >= 0.9)
    test("/version sha256", c.get("/version").json()["sha256"] == S["meta"]["sha256"])
    t0 = time.perf_counter()
    for x in Xte[:100]:
        c.post("/v1/predict", json={"belgilar": x.tolist()}, headers=K)
    rps = 100 / (time.perf_counter() - t0)
    m = c.get("/metrics").text
    test("metrics: predictions_total", "predictions_total 133" in m)

print("\n=== tumor-api testlari ===")
for nom, h in natija:
    print(f"{h:4s}  {nom}")
print(f"Jami: {sum(h == 'PASS' for _, h in natija)}/{len(natija)} PASS;  ketma-ket ~{rps:.0f}
so'rov/s (TestClient, 1 jarayon)")
print("\n--- /metrics ---\n" + m)
print("OpenAPI yo'llari:", ", ".join(sorted(app.openapi()["paths"])))
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== tumor-api testlari ===`, `Jami: 11/11 PASS`, `predictions_total`, `OpenAPI yo'llari`

TAJRIBA QILIB KO‘RING

- MODEL_VERSION ni os.environ orqali 'v2' qiling — /version va x-model-version sarlavhasi o'zgarishini tekshiring.
- Model faylini yozib bo'lgach bir baytini o'zgartiring: xizmat ishga tushmasligi (butunlik tekshiruvi) kerak.
- /metrics ga model bo'yicha so'rovlar sonini (label bilan) qo'shing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Konfiguratsiya qatlamlari, chuqur readiness, probe'lar va to'g'ri to'xtash

MAQSAD

(a) Sozlamalar uch qatlamdan yig'iladi: standart < ConfigMap (YAML) < muhit o'zgaruvchilari (TUMOR_*); har kalit sxema bo'yicha tur va diapazonga tekshiriladi, sir maskalanadi, xato bo'lsa xizmat ishga tushmaydi (fail fast); konfiguratsiya xeshi Deployment annotatsiyasi uchun hisoblanadi. (b) /readyz model fayli, uning xeshi, disk, xotira, o'z-o'zini sinash va drain holatini tekshiradi; /livez esa bog'liqliklarni tekshirmaydi. (c) Kubelet probe'lari simulyatsiyasi: model 45 s yuklanganda noto'g'ri liveness yoki qisqa startupProbe CrashLoopBackOff'ga olib kelishi ko'rsatiladi. (d) preStop va terminationGracePeriod ning yo'qolgan so'rovlarga ta'siri.

> **Kutilgan natija:** To'g'ri konfiguratsiyada OK va config-hash, xatoda 3 ta xato; buzilgan faylda readyz 503, livez 200; 2 ta XATO konfiguratsiyada 6 restart va hech qachon tayyor emas, to'g'rilarida 45 s; preStop=0 da ~300 so'rov yo'qoladi.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Konfiguratsiya qatlamlari va sog'liq tekshiruvlari: sozlamalar validatsiyasi, chuqur
readiness,
# kubelet probe'lari simulyatsiyasi (CrashLoopBackOff) va to'g'ri to'xtash (graceful shutdown)
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, shutil, time
import psutil, yaml
from fastapi import FastAPI
from fastapi.testclient import TestClient

# ---------------- (a) Konfiguratsiya: standart < ConfigMap (YAML) < muhit o'zgaruvchilari ----
------------
STANDART = {"model_version": "v1", "max_batch": 32, "log_level": "INFO", "workers": 2,
"timeout_s": 5.0, "api_key": ""}
SXEMA = {  # kalit: (tur, tekshiruv, izoh)
    "model_version": (str, lambda v: v in ("v1", "v2", "v3"), "v1|v2|v3"),
    "max_batch": (int, lambda v: 1 <= v <= 256, "1..256"),
    "log_level": (str, lambda v: v in ("DEBUG", "INFO", "WARNING", "ERROR"), "log darajasi"),
    "workers": (int, lambda v: 1 <= v <= 8, "1..8 (CPU limitiga mos)"),
    "timeout_s": (float, lambda v: 0.1 <= v <= 60, "0,1..60 s"),
    "api_key": (str, lambda v: len(v) >= 16, ">= 16 belgi (Secret)"),
}
CONFIGMAP = """
model_version: v2
max_batch: 64
log_level: INFO
workers: 2
"""


def yukla(configmap_text, env):
    cfg = dict(STANDART); manba = {k: "standart" for k in cfg}
    for k, v in (yaml.safe_load(configmap_text) or {}).items():
        cfg[k] = v; manba[k] = "ConfigMap"
    for k in cfg:
        e = env.get("TUMOR_" + k.upper())
```

```
        if e is not None:
            cfg[k] = e; manba[k] = "env"
    xatolar = []
    for k, (tur, shart, izoh) in SXEMA.items():
        try:
            cfg[k] = tur(cfg[k])
            if not shart(cfg[k]): xatolar.append(f"{k}={cfg[k]!r} — ruxsat: {izoh}")
        except (TypeError, ValueError):
            xatolar.append(f"{k}={cfg[k]!r} — tur {tur.__name__} emas")
    return cfg, manba, xatolar


def maskala(cfg):
    return {k: ("***" + v[-3:] if k == "api_key" and v else v) for k, v in cfg.items()}


print("=== (a) Konfiguratsiya qatlamlari ===")
for nom, env in (("to'g'ri", {"TUMOR_API_KEY": "s3cr3t-kalit-2026-xyz", "TUMOR_MAX_BATCH":
"128"}),
                 ("xato", {"TUMOR_API_KEY": "qisqa", "TUMOR_WORKERS": "ikki",
"TUMOR_TIMEOUT_S": "120"})):
    cfg, manba, xato = yukla(CONFIGMAP, env)
    print(f"\n[{nom}] yakuniy sozlama: {json.dumps(maskala(cfg), ensure_ascii=False)}")
    print("  manbalar: " + ", ".join(f"{k}<-{m}" for k, m in manba.items()))
    if xato:
        print("  XATO — xizmat ishga TUSHMAYDI (fail fast):"); [print("   -", x) for x in xato]
    else:
        h = hashlib.sha256(json.dumps(cfg, sort_keys=True).encode()).hexdigest()[:10]
        print(f"  OK. config-hash = {h}  -> Deployment annotatsiyasi: ConfigMap o'zgarsa
Pod'lar qayta yaratiladi")

# ---------------- (b) Chuqur readiness: bog'liqliklar tekshiruvi ----------------
os.makedirs("/tmp/models/v2", exist_ok=True)
open("/tmp/models/v2/model.pkl", "wb").write(b"demo-model-bytes")
KUTILGAN = hashlib.sha256(b"demo-model-bytes").hexdigest()
app = FastAPI()
HOLAT = {"draining": False}


def tekshiruvlar():
    t = {}
    p = "/tmp/models/v2/model.pkl"
    t["model_fayl"] = os.path.exists(p)
    t["model_sha"] = t["model_fayl"] and hashlib.sha256(open(p, "rb").read()).hexdigest() ==
KUTILGAN
    t["disk_bosh_MB>50"] = shutil.disk_usage("/tmp").free / 2 ** 20 > 50
    t["xotira_RSS_MB<1200"] = psutil.Process().memory_info().rss / 2 ** 20 < 1200
    t0 = time.perf_counter(); sum(i * i for i in range(20000)); t["selftest_ms<50"] =
(time.perf_counter() - t0) * 1e3 < 50
    t["draining_emas"] = not HOLAT["draining"]
    return t


@app.get("/livez")
def livez():                      # liveness — faqat jarayon tirikmi (bog'liqliklarni
TEKSHIRMAYDI!)
    return {"ok": True}


@app.get("/readyz")
def readyz():
    t = tekshiruvlar(); ok = all(t.values())
    from fastapi.responses import JSONResponse
```

```
    return JSONResponse({"ok": ok, "tekshiruvlar": t}, status_code=200 if ok else 503)


print("\n=== (b) Chuqur readiness ===")
with TestClient(app) as c:
    r = c.get("/readyz"); print("normal:      ", r.status_code, r.json()["tekshiruvlar"])
    open("/tmp/models/v2/model.pkl", "wb").write(b"buzilgan")
    r = c.get("/readyz"); print("buzilgan fayl:", r.status_code, {k: v for k, v in r.json()
["tekshiruvlar"].items() if not v})
    print("livez buzilganda ham:", c.get("/livez").status_code, "(to'g'ri: qayta ishga
tushirish muammoni hal qilmaydi)")
    open("/tmp/models/v2/model.pkl", "wb").write(b"demo-model-bytes")
    HOLAT["draining"] = True
    print("SIGTERM (drain):", c.get("/readyz").status_code)

# ---------------- (c) Kubelet probe simulyatsiyasi ----------------
def kubelet(startup_s, probe, T=600):
    """Konteyner startup_s soniyada tayyor bo'ladi. Probe'lar har 'period' soniyada
tekshiriladi.
    liveness/startup ketma-ket failureThreshold marta yiqilsa -> restart; kutish (backoff) 10,
20, 40 ... 300 s."""
    t, restart, backoff, voqealar = 0, 0, 10, []
    while t < T:
        bosh = t
        if "startup" in probe:                          # startupProbe o'tguncha liveness
ishlamaydi
            per, thr = probe["startup"]
            yiqildi_t = per * thr if startup_s > per * thr else None
            nom = "startupProbe"
        else:
            per, thr = probe["liv"]; d = probe["liv_delay"]
            erta = [d + per * i for i in range(thr)]    # ilova tayyor bo'lguncha bo'ladigan
birinchi thr ta tekshiruv
            yiqildi_t = erta[-1] if erta[-1] < startup_s else None
            nom = f"liveness {thr}x"
        if yiqildi_t is not None:
            t = bosh + yiqildi_t; restart += 1
            voqealar.append(f"t={t:4.0f}s {nom} yiqildi -> restart #{restart}
(CrashLoopBackOff, kutish {backoff}s)")
            t += backoff; backoff = min(backoff * 2, 300); continue
        rper = probe["rdy"]
        tayyor = bosh + rper * -(-startup_s // rper)     # birinchi muvaffaqiyatli readiness
tekshiruvi
        voqealar.append(f"t={tayyor:4.0f}s readiness OK -> Service trafikni yubora boshladi")
        return tayyor, restart, voqealar
    return None, restart, voqealar


KONF = {  # liv: (period, failureThreshold), rdy: readiness period
    "XATO: liveness 10s, startupProbe yo'q": {"liv_delay": 10, "liv": (5, 3), "rdy": 5},
    "XATO: startupProbe 2s x 15 (30 s)":    {"startup": (2, 15), "liv": (10, 3), "rdy": 5},
    "TO'G'RI: startupProbe 2s x 30":         {"startup": (2, 30), "liv": (10, 3), "rdy": 5},
    "TO'G'RI: liveness delay 60s":           {"liv_delay": 60, "liv": (10, 3), "rdy": 5},
}
print("\n=== (c) Kubelet probe simulyatsiyasi: model yuklanishi 45 s ===")
for nom, pr in KONF.items():
    tayyor, rs, v = kubelet(45, pr)
    print(f"\n{nom}: tayyor = {tayyor if tayyor is not None else 'HECH QACHON (600 s ichida)'},
restartlar = {rs}")
    for x in v[:5]: print("   " + x)

# ---------------- (d) Graceful shutdown: preStop + terminationGracePeriod ----------------
print("\n=== (d) To'xtashda yo'qolgan so'rovlar (100 so'rov/s, so'rov 0,4 s gacha) ===")
```

```
for prestop, grace in ((0, 30), (5, 30), (5, 3), (10, 30)):
    # endpoint yangilanishi (kube-proxy/ingress) ~3 s kechikadi: shu vaqt ichida kelgan
so'rovlar preStop bo'lmasa yo'qoladi
    kechikish = 3.0
    yoqol_kirish = 100 * max(0.0, kechikish - prestop)
    yoqol_jarayon = 100 * 0.4 if grace < prestop + 0.4 else 0
    print(f"preStop={prestop:2d}s grace={grace:2d}s -> yo'qolgan ~{yoqol_kirish +
yoqol_jarayon:5.0f} so'rov")
print("\n=== Konfiguratsiya va sog'liq tekshiruvlari tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) Konfiguratsiya qatlamlari ===`, `=== (b) Chuqur readiness ===`, `=== (c) Kubelet probe simulyatsiyasi`, `=== Konfiguratsiya va sog'liq tekshiruvlari tugadi ===`

TAJRIBA QILIB KO‘RING

- Sxemaga 'redis_url' (redis:// bilan boshlanishi shart) kalitini qo'shing va xato holatni sinang.
- Model yuklanishini 90 s qiling — startupProbe parametrlarini qanday tanlaysiz?
- Endpoint yangilanish kechikishini 3 s dan 8 s ga oshiring: preStop qancha bo'lishi kerak?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Dockerfile va docker-compose: generatsiya, validatsiya va qatlam keshi

MAQSAD

Loyiha parametrlaridan ko'p bosqichli (multi-stage) Dockerfile generatsiya qilinadi: builder bosqichida wheel'lar, yakuniy obrazda faqat runtime, root bo'lmagan foydalanuvchi, HEALTHCHECK va exec-shakldagi CMD. 13 qoidali tahlilchi uni ataylab yomon yozilgan Dockerfile bilan solishtiradi. Qatlam keshi xeshlar zanjiri bilan simulyatsiya qilinadi: faqat kod o'zgarganda nechta qatlam qayta qurilishi va yomon tartibda pip ham qayta ishlashi ko'rsatiladi. So'ng API + Redis + Prometheus uchun docker-compose.yml pyyaml bilan yaratiladi va tekshiriladi: obraz teglari, port to'qnashuvi, depends_on + healthcheck, hajm va tarmoqlar e'loni, sirlar, ishga tushish tartibi (topologik saralash) va sikl.

> **Kutilgan natija:** Yaxshi Dockerfile 13/13, yomoni 0/13; kod o'zgarganda 1 qatlam (~3 s), requirements o'zgarganda 5 qatlam (~200 s, taxminiy); compose OK, tartib redis -> api -> prometheus; buzilgan compose'da 4 xato.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Dockerfile va docker-compose: shablondan generatsiya, qoidalar bo'yicha validatsiya
va qatlam keshi simulyatsiyasi
# Docker sandbox'da yo'q — fayllar matn sifatida yaratiladi va Python'da tahlil qilinadi
(hadolint/compose config o'rniga).
import hashlib, re
import yaml

LOYIHA = {"nom": "tumor-api", "python": "3.12-slim-bookworm", "port": 8000, "uid": 10001,
          "paketlar": ["fastapi==0.115.6", "uvicorn==0.34.0", "scikit-learn==1.6.1",
"numpy==2.2.1"]}


def dockerfile(p):
    """Ko'p bosqichli (multi-stage) Dockerfile: builder'da g'ildiraklar (wheels), yakuniy
obrazda faqat runtime."""
    return f"""# syntax=docker/dockerfile:1
FROM python:{p['python']} AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

FROM python:{p['python']}
LABEL org.opencontainers.image.title="{p['nom']}" org.opencontainers.image.version="1.0.0"
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 MODEL_DIR=/models
RUN groupadd -g {p['uid']} app && useradd -u {p['uid']} -g app -s /usr/sbin/nologin app
WORKDIR /app
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
COPY --chown=app:app app/ ./app/
USER app
EXPOSE {p['port']}
HEALTHCHECK --interval=15s --timeout=3s --start-period=40s CMD python -c "import
urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://127.0.0.1:
{p['port']}/readyz').status==200 else 1)"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "{p['port']}", "--workers", "2"]
```

```
"""


YOMON = """FROM python:latest
ENV API_KEY=maxfiy123
COPY . .
RUN pip install fastapi uvicorn scikit-learn
RUN apt-get update
RUN apt-get install -y curl
EXPOSE 8080
CMD uvicorn app.main:app --port 8000
"""


def tahlil(txt, port):
    qatorlar = [l.strip() for l in txt.splitlines() if l.strip() and not
l.strip().startswith("#")]
    ins = [l.split()[0].upper() for l in qatorlar]
    froms = [l for l in qatorlar if l.upper().startswith("FROM")]
    q = {}
    q["D01 ko'p bosqichli (>=2 FROM)"] = len(froms) >= 2
    q["D02 bazaviy obraz teg bilan, ':latest' emas"] = all(":" in f.split()[1] and not
f.split()[1].endswith(":latest") for f in froms)
    q["D03 slim/distroless baza"] = any(k in froms[-1] for k in ("slim", "distroless",
"alpine"))
    q["D04 USER (root emas)"] = "USER" in ins and not any(re.match(r"USER\s+(root|0)\b", l) for
l in qatorlar)
    q["D05 ENV da sir yo'q"] = not any(re.search(r"ENV.*(KEY|SECRET|PASSWORD|TOKEN)=", l, re.I)
for l in qatorlar)
    q["D06 'COPY . .' yo'q"] = not any(re.match(r"COPY\s+\.\s+\.", l) for l in qatorlar)
    q["D07 requirements koddan oldin (kesh)"] = any("requirements" in l for l in qatorlar) and
\
        (min([i for i, l in enumerate(qatorlar) if "requirements" in l] or [99]) < max(i for i,
l in enumerate(qatorlar) if l.startswith("COPY")))
    q["D08 pip --no-cache-dir"] = all("--no-cache-dir" in l for l in qatorlar if "pip install"
in l)
    q["D09 versiyalar qotirilgan (==)"] = "requirements" in txt or all("==" in l for l in
qatorlar if "pip install" in l)
    q["D10 HEALTHCHECK bor"] = "HEALTHCHECK" in ins
    q["D11 EXPOSE = ilova porti"] = any(l == f"EXPOSE {port}" for l in qatorlar)
    q["D12 CMD exec shaklida (JSON)"] = any(l.startswith("CMD [") for l in qatorlar)
    q["D13 apt-get update alohida RUN emas"] = not any(l.strip() == "RUN apt-get update" for l
in qatorlar)
    return q


YAXSHI = dockerfile(LOYIHA)
print("=== Generatsiya qilingan Dockerfile ===\n" + YAXSHI)
print("=== Dockerfile qoidalari: yaxshi vs yomon ===")
qy, qn = tahlil(YAXSHI, 8000), tahlil(YOMON, 8000)
for k in qy:
    print(f"{k:42s} {'PASS' if qy[k] else 'FAIL':5s} | {'PASS' if qn[k] else 'FAIL'}")
print(f"Ball: yaxshi {sum(qy.values())}/{len(qy)}, yomon {sum(qn.values())}/{len(qn)}")

# --- Qatlam keshi simulyatsiyasi: har ko'rsatma xeshi = oldingi xesh + ko'rsatma + tegishli
fayllar xeshi ---
def qurish(txt, fayllar, kesh):
    """Har FROM yangi zanjir boshlaydi; COPY xeshiga fayllar mazmuni, COPY --from ga esa o'sha
bosqich xeshi qo'shiladi."""
    h = "0"; bosqich = {}; joriy = None; qayta = []
    for l in [l for l in txt.splitlines() if l.strip() and not l.startswith("#")]:
        soz = l.split()
        if soz[0] == "FROM":
```

```
            if joriy: bosqich[joriy] = h
            h = hashlib.sha1(l.encode()).hexdigest(); joriy = soz[-1] if "AS" in soz else
"yakuniy"; continue
        manba = ""
        if soz[0] == "COPY":
            argl = [a for a in soz[1:] if not a.startswith("--")]
            fr = next((a.split("=")[1] for a in soz[1:] if a.startswith("--from=")), None)
            if fr:
                manba = bosqich.get(fr, "")
            else:
                src = argl[0].rstrip("/").lstrip("./")
                manba = "".join(v for k, v in sorted(fayllar.items()) if src == "" or
k.startswith(src))
        h = hashlib.sha1((h + l + manba).encode()).hexdigest()
        if h not in kesh and soz[0] in ("RUN", "COPY"):
            qayta.append(soz[0] + " " + " ".join(a for a in soz[1:] if not a.startswith("--
chown"))[:30])
            kesh.add(h)
    return qayta


VAQT = {"RUN pip": 95, "RUN groupadd": 2}            # taxminiy qatlam qurish vaqti, s
fayllar = {"requirements.txt": "fastapi==0.115.6...", "app/main.py": "v1-kod", "app/model.py":
"m"}
kesh = set(); qurish(YAXSHI, fayllar, kesh)
print("\n=== Qatlam keshi (multi-stage, requirements alohida) ===")
for nom, ozgarish in (("faqat kod o'zgardi (app/main.py)", {"app/main.py": "v2-kod"}),
                      ("requirements o'zgardi", {"requirements.txt": "fastapi==0.116.0..."})):
    f2 = {**fayllar, **ozgarish}
    q = qurish(YAXSHI, f2, set(kesh))
    t = sum(next((v for k, v in VAQT.items() if x.startswith(k)), 3) for x in q)
    print(f"{nom:36s}: qayta quriladi {len(q)} qatlam ~{t:4d} s -> {q}")
kesh_y = set(); qurish(YOMON, fayllar, kesh_y)
q = qurish(YOMON, {**fayllar, "app/main.py": "v2-kod"}, set(kesh_y))
print(f"{'YOMON Dockerfile, faqat kod o`zgardi':36s}: qayta quriladi {len(q)} qatlam -> pip ham
qayta (COPY . . birinchi)")

# --- docker-compose: API + Redis kesh + Prometheus; generatsiya va validatsiya ---
COMPOSE = {
    "services": {
        "api": {"build": ".", "image": "registry.local/tumor-api:1.0.0", "ports":
["8000:8000"],
                "environment": {"MODEL_VERSION": "v1", "REDIS_URL": "redis://redis:6379/0"},
                "env_file": [".env"], "volumes": ["models:/models:ro"], "read_only": True,
                "depends_on": {"redis": {"condition": "service_healthy"}},
                "healthcheck": {"test": ["CMD", "python", "-c", "import
urllib.request;urllib.request.urlopen('http://127.0.0.1:8000/readyz')"],
                                "interval": "15s", "timeout": "3s", "retries": 3,
"start_period": "40s"},
                "deploy": {"resources": {"limits": {"cpus": "2", "memory": "1g"}}}, "restart":
"unless-stopped",
                "networks": ["ichki"]},
        "redis": {"image": "redis:7.4-alpine", "healthcheck": {"test": ["CMD", "redis-cli",
"ping"], "interval": "5s", "retries": 5},
                  "networks": ["ichki"], "restart": "unless-stopped"},
        "prometheus": {"image": "prom/prometheus:v3.1.0", "ports": ["9090:9090"], "volumes":
["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"],
                       "depends_on": {"api": {"condition": "service_healthy"}}, "networks":
["ichki"]},
    },
    "volumes": {"models": {}},
    "networks": {"ichki": {}},
}
```

```
matn = yaml.safe_dump(COMPOSE, sort_keys=False, allow_unicode=True)
print("\n=== docker-compose.yml (generatsiya) — birinchi 12 qator ===\n" +
"\n".join(matn.splitlines()[:12]) + "\n  ...")


def compose_tekshir(c):
    x = []; s = c["services"]; portlar = {}
    for nom, v in s.items():
        if ":" not in v.get("image", ":") or v.get("image", "").endswith(":latest"):
x.append(f"{nom}: obraz tegi yo'q/latest")
        for p in v.get("ports", []):
            h = p.split(":")[0]
            if h in portlar: x.append(f"{nom}: xost porti {h} band ({portlar[h]})")
            portlar[h] = nom
        for d, shart in (v.get("depends_on") or {}).items():
            if d not in s: x.append(f"{nom}: depends_on noma'lum xizmat {d}")
            elif shart.get("condition") == "service_healthy" and "healthcheck" not in s[d]:
x.append(f"{nom}: {d} da healthcheck yo'q")
        for vol in v.get("volumes", []):
            n = vol.split(":")[0]
            if not n.startswith(".") and not n.startswith("/") and n not in c.get("volumes",
{}): x.append(f"{nom}: hajm {n} e'lon qilinmagan")
        for n in v.get("networks", []):
            if n not in c.get("networks", {}): x.append(f"{nom}: tarmoq {n} e'lon qilinmagan")
        if any(re.search("KEY|SECRET|PASS", k) for k in (v.get("environment") or {})):
x.append(f"{nom}: sir environment ichida")
    # ishga tushish tartibi (topologik saralash) va sikl tekshiruvi
    tartib, qoldi = [], dict((k, set((v.get("depends_on") or {}).keys())) for k, v in
s.items())
    while qoldi:
        tayyor = [k for k, d in qoldi.items() if not d - set(tartib)]
        if not tayyor: x.append("depends_on sikli!"); break
        tartib += sorted(tayyor); [qoldi.pop(k) for k in tayyor]
    return x, tartib


x, tartib = compose_tekshir(yaml.safe_load(matn))
print("Compose validatsiya:", "OK" if not x else x, "| ishga tushish tartibi:", " ->
".join(tartib))
buz = yaml.safe_load(matn)
buz["services"]["prometheus"]["ports"] = ["8000:9090"]; buz["services"]
["redis"].pop("healthcheck")
buz["services"]["api"]["environment"]["API_KEY"] = "123"; buz["services"]["redis"]
["depends_on"] = {"prometheus": {}}
x, _ = compose_tekshir(buz)
print("Buzilgan compose xatolari:"); [print("  -", e) for e in x]
print("\n=== Dockerfile va compose tekshiruvi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Dockerfile qoidalari`, `=== Qatlam keshi`, `Compose validatsiya: OK`, `=== Dockerfile va compose tekshiruvi tugadi ===`

TAJRIBA QILIB KO‘RING

- Dockerfile'ga .dockerignore generatsiyasini va uni tekshiruvchi qoidani qo'shing (.git, __pycache__, *.ipynb).
- Compose'ga GPU ishlatadigan 'trainer' xizmatini (deploy.resources.reservations.devices) qo'shing va validatorni kengaytiring.
- Yomon Dockerfile'ni bosqichma-bosqich tuzatib, har tuzatishdan keyin ballni yozib boring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Kubernetes manifestlari: generator, dev/prod overlay va 18 qoidali tekshiruv

MAQSAD

Python generatori bitta parametrlar lug'atidan 9 ta obyekt yaratadi: Namespace, ConfigMap, Secret, Deployment (startup/readiness/liveness probe, preStop, securityContext, topologySpread, checksum/config annotatsiyasi, NAS'dagi PVC), Service, HPA (autoscaling/v2), PodDisruptionBudget, Ingress (TLS) va NetworkPolicy. Kustomize'ga o'xshash chuqur birlashtirish bilan dev va prod overlay'lari olinadi, natija multi-hujjat YAML'ga yozilib qayta o'qiladi. Tekshiruvchi obyekt qoidalari bilan birga obyektlararo moslikni ham ko'radi: Service selector, targetPort, Ingress backend, HPA target, PDB, configMapKeyRef kalitlari. Ataylab buzilgan manifestda tipik xatolar ushlanadi.

> **Kutilgan natija:** dev 8 obyekt (1 nusxada PDB yo'q), prod 9 obyekt; dev va prod 18/18 PASS, buzilgan 11/18; Ingress YAML ko'rsatiladi.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Kubernetes manifestlari: Python generatori (dev/prod overlay), pyyaml bilan multi-
hujjat YAML,
# obyekt qoidalari va obyektlararo moslik tekshiruvi (Service selector, Ingress backend, HPA
target, ConfigMap kalitlari)
import copy, hashlib, json
import yaml

ASOS = {"nom": "tumor-api", "ns": "ai-prod", "obraz": "registry.local/tumor-api:1.0.0", "port":
8000,
        "replicas": 3, "cpu_req": "500m", "cpu_lim": "1", "mem_req": "512Mi", "mem_lim": "1Gi",
        "hpa": (3, 10, 65), "host": "tumor.ai.local", "config": {"MODEL_VERSION": "v1",
"MAX_BATCH": "32", "LOG_LEVEL": "INFO"}}
OVERLAY = {"dev": {"ns": "ai-dev", "replicas": 1, "hpa": (1, 2, 80), "host": "tumor-
dev.ai.local", "config": {"LOG_LEVEL": "DEBUG"}},
           "prod": {}}


def birlashtir(a, b):
    """Kustomize'ga o'xshash chuqur birlashtirish (deep merge)."""
    r = copy.deepcopy(a)
    for k, v in b.items():
        r[k] = birlashtir(r[k], v) if isinstance(v, dict) and isinstance(r.get(k), dict) else v
    return r


def generatsiya(p):
    L = {"app": p["nom"], "app.kubernetes.io/version": p["obraz"].split(":")[-1]}
    cfg_hash = hashlib.sha256(json.dumps(p["config"], sort_keys=True).encode()).hexdigest()
[:10]
    meta = lambda nom, **kw: {"name": nom, "namespace": p["ns"], "labels": dict(L), **kw}
    hpa_min, hpa_max, nishon = p["hpa"]
    docs = [
        {"apiVersion": "v1", "kind": "Namespace", "metadata": {"name": p["ns"]}},
        {"apiVersion": "v1", "kind": "ConfigMap", "metadata": meta(p["nom"] + "-config"),
"data": p["config"]},
        {"apiVersion": "v1", "kind": "Secret", "metadata": meta(p["nom"] + "-secret"), "type":
"Opaque",
```

```
         "stringData": {"API_KEY": "<<CI/CD da Vault'dan qo'yiladi>>"}},
        {"apiVersion": "apps/v1", "kind": "Deployment", "metadata": meta(p["nom"]),
         "spec": {"replicas": p["replicas"], "revisionHistoryLimit": 5,
                  "strategy": {"type": "RollingUpdate", "rollingUpdate": {"maxSurge": 1,
"maxUnavailable": 0}},
                  "selector": {"matchLabels": {"app": p["nom"]}},
                  "template": {"metadata": {"labels": dict(L), "annotations":
{"checksum/config": cfg_hash}},
                               "spec": {"securityContext": {"runAsNonRoot": True, "runAsUser":
10001, "seccompProfile": {"type": "RuntimeDefault"}},
                                        "terminationGracePeriodSeconds": 30,
                                        "topologySpreadConstraints": [{"maxSkew": 1,
"topologyKey": "kubernetes.io/hostname",
                                                                       "whenUnsatisfiable":
"ScheduleAnyway", "labelSelector": {"matchLabels": {"app": p["nom"]}}}],
                                        "containers": [{
                                            "name": "api", "image": p["obraz"],
"imagePullPolicy": "IfNotPresent",
                                            "ports": [{"name": "http", "containerPort":
p["port"]}],
                                            "envFrom": [{"configMapRef": {"name": p["nom"] + "-
config"}}],
                                            "env": [{"name": "API_KEY", "valueFrom":
{"secretKeyRef": {"name": p["nom"] + "-secret", "key": "API_KEY"}}},
                                                    {"name": "MODEL_VERSION", "valueFrom":
{"configMapKeyRef": {"name": p["nom"] + "-config", "key": "MODEL_VERSION"}}}],
                                            "resources": {"requests": {"cpu": p["cpu_req"],
"memory": p["mem_req"]}, "limits": {"cpu": p["cpu_lim"], "memory": p["mem_lim"]}},
                                            "startupProbe": {"httpGet": {"path": "/startupz",
"port": "http"}, "periodSeconds": 2, "failureThreshold": 30},
                                            "readinessProbe": {"httpGet": {"path": "/readyz",
"port": "http"}, "periodSeconds": 5},
                                            "livenessProbe": {"httpGet": {"path": "/livez",
"port": "http"}, "periodSeconds": 10, "failureThreshold": 3},
                                            "lifecycle": {"preStop": {"exec": {"command":
["sleep", "5"]}}},
                                            "securityContext": {"readOnlyRootFilesystem": True,
"allowPrivilegeEscalation": False, "capabilities": {"drop": ["ALL"]}},
                                            "volumeMounts": [{"name": "models", "mountPath":
"/models", "readOnly": True}]}],
                                        "volumes": [{"name": "models", "persistentVolumeClaim":
{"claimName": "models-nas", "readOnly": True}}]}}}},
        {"apiVersion": "v1", "kind": "Service", "metadata": meta(p["nom"]),
         "spec": {"selector": {"app": p["nom"]}, "ports": [{"name": "http", "port": 80,
"targetPort": "http"}]}},
        {"apiVersion": "autoscaling/v2", "kind": "HorizontalPodAutoscaler", "metadata":
meta(p["nom"]),
         "spec": {"scaleTargetRef": {"apiVersion": "apps/v1", "kind": "Deployment", "name":
p["nom"]},
                  "minReplicas": hpa_min, "maxReplicas": hpa_max,
                  "metrics": [{"type": "Resource", "resource": {"name": "cpu", "target":
{"type": "Utilization", "averageUtilization": nishon}}}],
                  "behavior": {"scaleDown": {"stabilizationWindowSeconds": 300}}}},
        {"apiVersion": "policy/v1", "kind": "PodDisruptionBudget", "metadata": meta(p["nom"]),
         "spec": {"minAvailable": max(1, p["replicas"] - 1), "selector": {"matchLabels":
{"app": p["nom"]}}}},
        {"apiVersion": "networking.k8s.io/v1", "kind": "Ingress",
         "metadata": meta(p["nom"], annotations={"nginx.ingress.kubernetes.io/proxy-body-size":
"1m"}),
         "spec": {"ingressClassName": "nginx", "tls": [{"hosts": [p["host"]], "secretName":
p["nom"] + "-tls"}],
                  "rules": [{"host": p["host"], "http": {"paths": [{"path": "/", "pathType":
"Prefix",
```

```
                                "backend": {"service": {"name": p["nom"], "port": {"name":
"http"}}}}]}}]},
        {"apiVersion": "networking.k8s.io/v1", "kind": "NetworkPolicy", "metadata":
meta(p["nom"] + "-faqat-ingress"),
         "spec": {"podSelector": {"matchLabels": {"app": p["nom"]}}, "policyTypes":
["Ingress"],
                  "ingress": [{"from": [{"namespaceSelector": {"matchLabels":
{"kubernetes.io/metadata.name": "ingress-nginx"}}}],
                               "ports": [{"port": p["port"]}]}]}},
    ]
    # 1 nusxali muhitda PDB tugun drain'ini butunlay bloklaydi — shuning uchun faqat replicas
>= 2 da yaratiladi
    return [h for h in docs if h["kind"] != "PodDisruptionBudget" or p["replicas"] >= 2]


def mcpu(v):
    return int(v[:-1]) if str(v).endswith("m") else int(float(v) * 1000)


def mib(v):
    return int(v[:-2]) * (1024 if v.endswith("Gi") else 1)


def tekshir(hujjatlar):
    q = {}; ob = {(h["kind"], h["metadata"]["name"]): h for h in hujjatlar}
    dep = next(h for h in hujjatlar if h["kind"] == "Deployment"); t = dep["spec"]["template"];
c = t["spec"]["containers"][0]
    q["K01 obraz tegi qotirilgan (latest emas)"] = ":" in c["image"] and not
c["image"].endswith(":latest")
    q["K02 requests/limits bor"] = all(k in c.get("resources", {}).get(x, {}) for x in
("requests", "limits") for k in ("cpu", "memory"))
    q["K03 requests <= limits"] = mcpu(c["resources"]["requests"]["cpu"]) <=
mcpu(c["resources"]["limits"]["cpu"]) and \
        mib(c["resources"]["requests"]["memory"]) <= mib(c["resources"]["limits"]["memory"])
    q["K04 startup/readiness/liveness probe"] = all(k in c for k in ("startupProbe",
"readinessProbe", "livenessProbe"))
    q["K05 readiness != liveness yo'li"] = c["readinessProbe"]["httpGet"]["path"] !=
c["livenessProbe"]["httpGet"]["path"]
    q["K06 runAsNonRoot + readOnlyRootFS"] = t["spec"].get("securityContext",
{}).get("runAsNonRoot") and c.get("securityContext", {}).get("readOnlyRootFilesystem")
    q["K07 sir Secret orqali (ochiq env emas)"] = not any("KEY" in e["name"] and "value" in e
for e in c.get("env", []))
    q["K08 selector == template labels"] = all(t["metadata"]["labels"].get(k) == v for k, v in
dep["spec"]["selector"]["matchLabels"].items())
    svc = ob.get(("Service", dep["metadata"]["name"]))
    q["K09 Service selector Pod'larga mos"] = bool(svc) and all(t["metadata"]["labels"].get(k)
== v for k, v in svc["spec"]["selector"].items())
    portnom = {p.get("name") for p in c["ports"]} | {p["containerPort"] for p in c["ports"]}
    q["K10 Service targetPort konteynerda bor"] = bool(svc) and all(p["targetPort"] in portnom
for p in svc["spec"]["ports"])
    ing = next((h for h in hujjatlar if h["kind"] == "Ingress"), None)
    be = [pp["backend"]["service"] for r in ing["spec"]["rules"] for pp in r["http"]["paths"]]
if ing else []
    q["K11 Ingress backend Service mavjud"] = bool(be) and all(("Service", b["name"]) in ob and
        any(b["port"].get("name") == sp.get("name") or b["port"].get("number") == sp["port"]
for sp in ob[("Service", b["name"])]["spec"]["ports"]) for b in be)
    q["K12 Ingress TLS bor"] = bool(ing and ing["spec"].get("tls"))
    hpa = next((h for h in hujjatlar if h["kind"] == "HorizontalPodAutoscaler"), None)
    q["K13 HPA target Deployment mavjud"] = bool(hpa) and ("Deployment", hpa["spec"]
["scaleTargetRef"]["name"]) in ob
    q["K14 HPA min <= replicas <= max"] = bool(hpa) and hpa["spec"]["minReplicas"] <=
dep["spec"]["replicas"] <= hpa["spec"]["maxReplicas"]
    pdb = next((h for h in hujjatlar if h["kind"] == "PodDisruptionBudget"), None)
```

```
    q["K15 PDB minAvailable < replicas (drain bloklanmaydi)"] = pdb is None or pdb["spec"]
["minAvailable"] < dep["spec"]["replicas"]
    cm_kalit = {(h["metadata"]["name"], k) for h in hujjatlar if h["kind"] == "ConfigMap" for k
in h.get("data", {})}
    ref = [e["valueFrom"]["configMapKeyRef"] for e in c.get("env", []) if "configMapKeyRef" in
e.get("valueFrom", {})]
    q["K16 configMapKeyRef kalitlari mavjud"] = all((r["name"], r["key"]) in cm_kalit for r in
ref)
    q["K17 checksum/config annotatsiyasi"] = "checksum/config" in
t["metadata"].get("annotations", {})
    q["K18 hamma obyekt bitta namespace'da"] = len({h["metadata"].get("namespace") for h in
hujjatlar if h["kind"] != "Namespace"}) == 1
    return q


natija = {}
for muhit, ov in OVERLAY.items():
    p = birlashtir(ASOS, ov)
    docs = generatsiya(p)
    matn = yaml.safe_dump_all(docs, sort_keys=False, allow_unicode=True)
    qayta = list(yaml.safe_load_all(matn))                # aylanma tekshiruv: YAML -> Python
    natija[muhit] = tekshir(qayta)
    print(f"[{muhit}] {len(qayta)} obyekt: " + ", ".join(h["kind"] for h in qayta) + f"; YAML
{len(matn.splitlines())} qator")
    if muhit == "prod":
        PROD = qayta
        print("\n--- Ingress (prod) ---\n" + yaml.safe_dump(next(h for h in qayta if h["kind"]
== "Ingress"), sort_keys=False))

# --- Ataylab buzilgan manifest: tipik xatolar ---
buz = copy.deepcopy(PROD)
d = next(h for h in buz if h["kind"] == "Deployment"); cc = d["spec"]["template"]["spec"]
["containers"][0]
cc["image"] = "registry.local/tumor-api:latest"; cc["resources"]["requests"]["cpu"] = "2"
cc["env"].append({"name": "API_KEY_OCHIQ", "value": "123"}); cc["env"][1]["valueFrom"]
["configMapKeyRef"]["key"] = "MODEL_VER"
next(h for h in buz if h["kind"] == "Service")["spec"]["selector"] = {"app": "tumor"}
next(h for h in buz if h["kind"] == "Ingress")["spec"]["rules"][0]["http"]["paths"][0]
["backend"]["service"]["port"] = {"number": 8080}
next(h for h in buz if h["kind"] == "PodDisruptionBudget")["spec"]["minAvailable"] = 3
natija["buzilgan"] = tekshir(buz)

print("=== Manifest qoidalari (dev / prod / buzilgan) ===")
for k in natija["prod"]:
    print(f"{k:52s} " + "  ".join(f"{'PASS' if natija[m][k] else 'FAIL':5s}" for m in natija))
for m, q in natija.items():
    print(f"{m:9s}: {sum(map(bool, q.values()))}/{len(q)} PASS")
print("\n=== K8s manifestlari tekshiruvi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Manifest qoidalari`, `prod : 18/18 PASS`, `=== K8s manifestlari tekshiruvi tugadi ===`

TAJRIBA QILIB KO‘RING

- 'staging' overlay qo'shing (replicas 2, boshqa host) va uchala muhitni bitta jadvalda solishtiring.
- HPA'ga ikkinchi metrika (Pods turidagi http_requests_per_second) qo'shing va validatorga uning tekshiruvini yozing.
- Generatsiya qilingan YAML'ni faylga yozib, real klasterda `kubectl apply --dry-run=server -f` bilan tekshiring (agar klaster bo'lsa).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Versiyani chiqarish: RollingUpdate simulyatsiyasi va haqiqiy so'rovlar bilan canary

MAQSAD

(a) Deployment kontrolleri 1 s qadamli modelda simulyatsiya qilinadi: 6 nusxa, har biri 50 so'rov/s, talab 260 so'rov/s, yangi Pod 30 s da tayyor. maxSurge/maxUnavailable kombinatsiyalari, blue/green va Recreate uchun yangilanish davomiyligi, minimal sig'im, yo'qolgan so'rovlar va maksimal Pod soni solishtiriladi. (b) Ikki FastAPI versiyasi TestClient orqali ishga tushadi va vaznli marshrutlovchi (Ingress/Service mesh o'rnida) trafikni 5 -> 25 -> 50 -> 100 % bosqichlarda bo'ladi. Har bosqichda kamida 60 canary namunasi yig'ilib, xato ulushi va p95 kechikish bazaviy versiya bilan solishtiriladi; mezon buzilsa avtomatik orqaga qaytariladi. Sog'lom, nuqsonli va sekin v2 sinab ko'riladi.

> **Kutilgan natija:** maxUnavailable=0 da yo'qotish 0, maxUnavailable=1 da ~1800, Recreate da ~7800 so'rov yo'qoladi; canary: sog'lom v2 100 % gacha chiqadi, nuqsonli va sekin v2 5 % bosqichda to'xtatiladi.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Yangi versiyani chiqarish: RollingUpdate simulyatsiyasi (maxSurge/maxUnavailable) va
# haqiqiy so'rovlar bilan canary — ikki FastAPI versiyasi, vaznli marshrutlovchi (Ingress
o'rnida) va avtomatik tahlil
import warnings; warnings.filterwarnings("ignore")
import math, random, time
import numpy as np
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel

# ---------------- (a) RollingUpdate simulyatsiyasi ----------------
def rolling(N=6, surge=1, unav=0, start_s=30, sig=50, talab=260, recreate=False, T=900):
    """1 s qadamli model. Har Pod sig' so'rov/s; yangi Pod start_s dan keyin tayyor. Deployment
kontrolleri:
    yangi Pod'lar jami <= N+surge bo'lguncha yaratiladi; eski Pod'lar mavjudlik >= N-unav
saqlangan holda o'chiriladi."""
    eski, yangi_tayyor, startda = N, 0, []
    tushdi = 0.0; min_sig = 10 ** 9; maks_pod = N; t = 0
    if recreate:
        eski = 0                                          # Recreate: hammasi birdan o'chadi
    while t < T:
        yangi_tayyor += sum(1 for r in startda if r == t); startda = [r for r in startda if r >
t]
        if not recreate:
            while eski > 0 and eski - 1 + yangi_tayyor >= N - unav:
                eski -= 1
        while eski + yangi_tayyor + len(startda) < N + (0 if recreate else surge) and
yangi_tayyor + len(startda) < N:
            startda.append(t + start_s)
        maks_pod = max(maks_pod, eski + yangi_tayyor + len(startda))
        sig_t = (eski + yangi_tayyor) * sig; min_sig = min(min_sig, sig_t)
        tushdi += max(0, talab - sig_t)
        if yangi_tayyor == N and eski == 0:
            return t, min_sig, tushdi, maks_pod
```

```
        t += 1
    return None, min_sig, tushdi, maks_pod


print("=== (a) RollingUpdate: 6 nusxa x 50 so'rov/s, talab 260 so'rov/s, Pod start 30 s ===")
print(f"{'strategiya':34s} {'davomiylik s':>12s} {'min sig`im':>11s} {'yo`qolgan so`rov':>17s}
{'maks Pod':>9s}")
for nom, kw in (("maxSurge=1, maxUnavailable=0", dict(surge=1, unav=0)), ("maxSurge=2,
maxUnavailable=0", dict(surge=2, unav=0)),
                ("maxSurge=0, maxUnavailable=1", dict(surge=0, unav=1)), ("maxSurge=25%(2),
maxUnav=25%(2)", dict(surge=2, unav=2)),
                ("maxSurge=6 (blue/green kabi)", dict(surge=6, unav=0)), ("Recreate",
dict(recreate=True))):
    d, ms, tu, mp = rolling(**kw)
    print(f"{nom:34s} {d:12d} {ms:11d} {tu:17.0f} {mp:9d}")
print("maxUnavailable>0 tezroq va arzonroq (kam qo'shimcha Pod), lekin sig'im talabdan pastga
tushsa so'rovlar yo'qoladi.")

# ---------------- (b) Canary: haqiqiy so'rovlar bilan ----------------
def xizmat(versiya, xato_ulush, qoshimcha_ms):
    app = FastAPI(); rng = random.Random(sum(map(ord, versiya)))   # takrorlanuvchi urug'

    class Kirish(BaseModel):
        x: list[float]

    @app.post("/predict")
    def predict(k: Kirish):
        t0 = time.perf_counter()
        s = float(np.tanh(np.array(k.x) @ np.linspace(-1, 1, len(k.x))))
        while (time.perf_counter() - t0) * 1e3 < qoshimcha_ms: pass     # sun'iy qo'shimcha
hisob
        if rng.random() < xato_ulush: raise HTTPException(500, "ichki xato")
        return {"versiya": versiya, "ball": s}
    return TestClient(app)


def canary(v1, v2, qadamlar=(5, 25, 50, 100), min_namuna=60, maks_sorov=1500, seed=1):
    rng = random.Random(seed); jurnal = []
    for w in qadamlar:
        st = {"v1": [], "v2": []}; n = 0
        while (len(st["v2"]) < min_namuna or (w < 100 and len(st["v1"]) < min_namuna)) and n <
maks_sorov:
            ver = "v2" if rng.random() * 100 < w else "v1"
            t0 = time.perf_counter()
            r = (v2 if ver == "v2" else v1).post("/predict", json={"x": [rng.random() for _ in
range(16)]})
            st[ver].append((r.status_code, (time.perf_counter() - t0) * 1e3)); n += 1
        e = {v: (np.mean([s >= 500 for s, _ in st[v]]) if st[v] else 0.0) for v in st}
        p95 = {v: (np.percentile([l for _, l in st[v]], 95) if st[v] else 0.0) for v in st}
        # qaror: canary xato ulushi bazadan 2 p.p. dan ortiq yoki p95 1,5x (va kamida +2 ms)
katta bo'lsa — orqaga qaytarish
        baza_e = e["v1"] if st["v1"] else jurnal[-1]["xato_v1_%"] / 100
        baza_p = p95["v1"] if st["v1"] else jurnal[-1]["p95_v1_ms"]
        yomon = e["v2"] > baza_e + 0.02 or p95["v2"] > max(1.5 * baza_p, baza_p + 2.0)
        jurnal.append({"vazn_%": w, "so'rov": n, "v2_namuna": len(st["v2"]), "xato_v1_%": 100 *
baza_e, "xato_v2_%": 100 * e["v2"],
                       "p95_v1_ms": baza_p, "p95_v2_ms": p95["v2"], "qaror": "ORQAGA QAYTARISH"
if yomon else "davom"})
        if yomon: break
    return pd.DataFrame(jurnal)


v1 = xizmat("v1", 0.005, 0.0)
```

```
for nom, v2 in (("v2 sog'lom (xato 0,5 %, +0 ms)", xizmat("v2", 0.005, 0.0)),
                ("v2 nuqsonli (xato 8 %)", xizmat("v2b", 0.08, 0.0)),
                ("v2 sekin (+6 ms hisob)", xizmat("v2c", 0.005, 6.0))):
    j = canary(v1, v2)
    print(f"\n=== (b) Canary: {nom} ===")
    print(j.to_string(index=False, float_format=lambda v: f"{v:.2f}"))
    print("Natija:", "v2 TO'LIQ CHIQARILDI (100 %)" if j.iloc[-1]["qaror"] == "davom" and
j.iloc[-1]["vazn_%"] == 100
          else f"v2 {int(j.iloc[-1]['vazn_%'])} % bosqichda to'xtatildi, trafik v1 ga
qaytarildi")
print("\nEslatma: 5 % bosqichda 60 ta canary namunasi uchun ~1200 so'rov kerak — kichik
trafikda canary sekin qaror qiladi.")
print("\n=== Rolling va canary tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) RollingUpdate`, `=== (b) Canary: v2 nuqsonli`, `=== Rolling va canary tugadi ===`

TAJRIBA QILIB KO‘RING

- Canary qaroriga ikki ulush uchun z-test qo'shing ($p < 0,05$) va qat'iy chegara bilan solishtiring.
- Pod start vaqtini 90 s qiling: qaysi strategiya maqbul bo'ladi?
- v2 xato ulushini 2,5 % qiling — canary uni qaysi bosqichda va necha so'rovdan keyin ushlaydi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. HPA (autoscaling/v2) + Cluster Autoscaler + Erlang-C: yuklama simulyatsiyasi

MAQSAD

HPA'ning haqiqiy algoritmi modellashtiriladi: desired = ceil(current x metrika / nishon), 10 % tolerantlik, bir nechta metrikada maksimum, scaleUp siyosati (15 s da 100 % yoki 4 Pod) va scaleDown uchun 300 s stabilizatsiya oynasi. Pod'lar tugunlarga joylashadi (tugunda 3 Pod), joy yetmasa Pending bo'ladi va Cluster Autoscaler yangi tugun so'raydi (120 s), ortiqcha tugunni 10 daqiqadan keyin olib tashlaydi. Kechikish M/M/c (Erlang C) bilan baholanadi. 6 soatlik kunlik to'lqin va 10 daqiqalik keskin sakrash (+450 so'rov/s) ostida on-prem (4 tugun) va bulut konfiguratsiyalari SLO buzilish daqiqalari, yo'qolgan so'rovlar, Pod-soat va tugun-soat bo'yicha solishtiriladi.

> **Kutilgan natija:** On-prem 4 tugunda sakrashda ~10 daqiqa SLO buzilishi va ~110 ming so'rov yo'qoladi; bulut CA bilan ~1,5-5 daqiqa; Erlang-C: 300 so'rov/s da 7 Pod p95 ~110 ms, 10 Pod ~64 ms.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. HPA (autoscaling/v2) algoritmi + Cluster Autoscaler + Erlang-C navbat modeli:
yuklama simulyatsiyasi
# HPA formulasi: desired = ceil(current * joriy_metrika / nishon), tolerantlik 10 %, bir nechta
metrika — maksimum.
# behavior: scaleUp — 15 s da max(100 %, 4 Pod); scaleDown — 300 s stabilizatsiya oynasi, 60 s
da 10 %.
# Pod 1 yadro so'raydi, 1 so'rov = 20 ms CPU -> Pod sig'imi ~50 so'rov/s; tugun 4 yadro (3,5
ajratiladi) -> 3 Pod. Kechikish — M/M/c (Erlang C).
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DT = 15                          # HPA sinxronlash davri, s
T = 6 * 3600                     # 6 soat
XIZMAT = 0.020                   # s/so'rov (CPU vaqti)
MU = 1 / XIZMAT                  # 50 so'rov/s/Pod
REQ_CPU = 1.0                    # Pod requests.cpu = limits.cpu = 1 yadro (Guaranteed QoS)
SLO_MS = 200


def yuklama(t):
    """Kunlik to'lqin (80 -> 320 so'rov/s) + t=3 soatda 10 daqiqalik keskin sakrash (aksiya,
+450)."""
    s = 80 + 240 * max(0.0, math.sin(math.pi * t / T)) ** 2
    if 3 * 3600 <= t < 3 * 3600 + 600: s += 450
    return s


def erlang_c(lam, c, mu):
    """M/M/c: navbatda kutish ehtimoli C va p95 javob vaqti (ms). Ortiqcha yuklamada None."""
    rho = lam / (c * mu)
    if rho >= 1: return None
    a = lam / mu; s = sum(a ** k / math.factorial(k) for k in range(c)); last = a ** c /
math.factorial(c) / (1 - rho)
```

```
    C = last / (s + last)
    kut = 0.0 if C <= 0.05 else math.log(C / 0.05) / (c * mu - lam)
    return 1e3 * (kut + XIZMAT * 3.0)                # xizmat vaqtining p95 ~ 3x o'rtacha
(eksponensial)


def sim(nom, min_r=2, max_r=30, cpu_nishon=70, rps_nishon=None, tugun_max=4, tugun_boshl=4,
tugun_start=120,
        pod_start=30, pod_per_tugun=3):
    tayyor = min_r; startda = []                      # (tayyor bo'lish vaqti)
    kutmoqda = 0                                      # Pending Pod'lar (tugunda joy yo'q)
    tugun = tugun_boshl; tugun_kelyapti = []
    tavsiya_tarix = []                                # scaleDown stabilizatsiya oynasi uchun
    oxirgi_down = -999; ortiqcha_s = 0
    log = []; slo_buzildi = 0; tushdi = 0.0; pod_s = 0.0; tugun_s = 0.0
    for t in range(0, T, DT):
        lam = yuklama(t)
        tayyor += sum(1 for r in startda if r <= t); startda = [r for r in startda if r > t]
        tugun += sum(1 for r in tugun_kelyapti if r <= t); tugun_kelyapti = [r for r in
tugun_kelyapti if r > t]
        # Pending Pod'larni joy bo'lsa joylash
        joy = tugun * pod_per_tugun - (tayyor + len(startda))
        joyla = min(joy, kutmoqda); kutmoqda -= joyla; startda += [t + pod_start] * joyla
        # HPA hisobi (faqat tayyor Pod'lar metrikasi)
        cpu_util = 100 * min(lam / tayyor * XIZMAT, 1.0) / REQ_CPU          # requests ga
nisbatan %
        joriy = tayyor + len(startda) + kutmoqda
        tavs = []
        for metr, nishon in (("cpu", cpu_nishon), ("rps", rps_nishon)):
            if nishon is None: continue
            qiymat = cpu_util if metr == "cpu" else lam / tayyor
            nisbat = qiymat / nishon
            tavs.append(joriy if abs(nisbat - 1) <= 0.1 else math.ceil(tayyor * nisbat))
        desired = min(max(max(tavs), min_r), max_r)
        tavsiya_tarix.append((t, desired)); tavsiya_tarix = [(a, b) for a, b in tavsiya_tarix
if t - a <= 300]
        if desired > joriy:
            desired = min(desired, joriy + max(joriy, 4))                    # scaleUp
siyosati
            kutmoqda += desired - joriy
        else:
            stabil = max(b for _, b in tavsiya_tarix)                        # oyna ichidagi
eng katta tavsiya
            if stabil < joriy and t - oxirgi_down >= 60:
                kamay = min(joriy - stabil, max(1, math.floor(0.1 * joriy)))
                ol = min(kamay, kutmoqda); kutmoqda -= ol; kamay -= ol
                tayyor -= kamay; oxirgi_down = t
        # Cluster Autoscaler: Pending bor bo'lsa yangi tugun so'raydi
        kerak = math.ceil(kutmoqda / pod_per_tugun) - len(tugun_kelyapti)
        if kerak > 0 and tugun + len(tugun_kelyapti) < tugun_max:
            tugun_kelyapti += [t + tugun_start] * min(kerak, tugun_max - tugun -
len(tugun_kelyapti))
        # CA scale-down: tugun 10 daqiqa ortiqcha bo'lsa (Pod'lar kamroq tugunga sig'sa) —
bittasi olib tashlanadi
        zarur = max(tugun_boshl, math.ceil((tayyor + len(startda) + kutmoqda) / pod_per_tugun))
        ortiqcha_s = ortiqcha_s + DT if tugun > zarur else 0
        if ortiqcha_s >= 600:
            tugun -= 1; ortiqcha_s = 0
        # SLO: kechikish va yo'qolgan so'rovlar
        p95 = erlang_c(lam, tayyor, MU)
        if p95 is None or p95 > SLO_MS:
            slo_buzildi += DT
        tushdi += max(0.0, lam - tayyor * MU * 0.98) * DT
```

```
        pod_s += (tayyor + len(startda)) * DT; tugun_s += tugun * DT
        log.append((t / 3600, lam, tayyor, kutmoqda, tugun))
    return {"konfiguratsiya": nom, "SLO_buzilish_daq": slo_buzildi / 60, "yo'qolgan_so'rov":
tushdi,
            "Pod_soat": pod_s / 3600, "tugun_soat": tugun_s / 3600, "maks_Pod": max(l[2] for l
in log)}, log


KONF = [
    ("asos: CPU 70 %, 4 tugun (on-prem)", {}),
    ("CPU 50 % (zaxira ko'p)", dict(cpu_nishon=50)),
    ("CPU 70 % + RPS 30/Pod", dict(rps_nishon=30)),
    ("bulut: CA 4 -> 10 tugun", dict(tugun_max=10)),
    ("bulut + RPS 30/Pod", dict(tugun_max=10, rps_nishon=30)),
    ("bulut + sekin tugun (300 s)", dict(tugun_max=10, tugun_start=300)),
    ("bulut + oldindan 6 tugun, min 8", dict(tugun_max=10, tugun_boshl=6, min_r=8)),
]
natija = []; loglar = {}
for nom, kw in KONF:
    r, lg = sim(nom, **kw); natija.append(r); loglar[nom] = lg
df = pd.DataFrame(natija)
print("=== HPA + Cluster Autoscaler simulyatsiyasi (6 soat, 15 s qadam) ===")
print("Yuklama: 80 -> 320 so'rov/s kunlik to'lqin + 3-soatda 10 daqiqa +450 so'rov/s; Pod
sig'imi 50 so'rov/s; 1 tugun = 3 Pod")
print(df.to_string(index=False, float_format=lambda v: f"{v:.1f}"))

print("Kuzating: CPU metrikasi 100 % da to'yinadi (Pod o'z sig'imidan ko'p ishlay olmaydi),
shuning uchun HPA bir qadamda")
print("faqat ~1,4x o'sadi; RPS metrikasi haqiqiy talabni ko'radi, lekin asosiy cheklov — tugun
(120 s) va Pod (30 s) start vaqti.")
print("On-prem'da 4 tugun (12 Pod = 600 so'rov/s) sakrashni ko'tarmaydi; oldindan zaxira eng
kam buzilish beradi, ammo qimmatroq.")
print("\n=== Erlang-C: 300 so'rov/s da Pod soniga qarab p95 (ms) ===")
for c in (7, 8, 9, 10, 12):
    p = erlang_c(300, c, MU)
    print(f"c={c:2d}: foydalanish {300 / (c * MU) * 100:5.1f} %, p95 = {'ortiqcha yuklama' if p
is None else f'{p:6.1f} ms'}")

fig, ax = plt.subplots(2, 1, figsize=(9, 5.5), sharex=True)
lg = np.array(loglar[KONF[0][0]]); lg2 = np.array(loglar[KONF[4][0]])
ax[0].plot(lg[:, 0], lg[:, 1], "k-"); ax[0].plot(lg[:, 0], lg[:, 2] * MU, "b--", label="sig'im
(asos)")
ax[0].plot(lg2[:, 0], lg2[:, 2] * MU, "g:", label="sig'im (bulut CA)");
ax[0].set_ylabel("so'rov/s"); ax[0].legend()
ax[1].step(lg[:, 0], lg[:, 2], label="tayyor Pod (asos)"); ax[1].step(lg2[:, 0], lg2[:, 2],
label="tayyor Pod (bulut)")
ax[1].step(lg2[:, 0], lg2[:, 4], label="tugunlar (bulut)"); ax[1].set_xlabel("vaqt, soat");
ax[1].legend(fontsize=8)
plt.tight_layout()
print("\n=== HPA simulyatsiyasi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== HPA + Cluster Autoscaler simulyatsiyasi`, `=== Erlang-C`, `=== HPA simulyatsiyasi tugadi ===`

TAJRIBA QILIB KO‘RING

- scaleUp siyosatini 15 s da 2 Pod ga cheklang — SLO buzilishi qanday o'zgaradi?
- Sakrashni prognoz qilib (aksiya jadvali), t=2:55 da minReplicas ni vaqtincha 16 ga ko'taring (KEDA cron kabi).

- Pod start vaqtini 30 s dan 10 s ga tushirsangiz (kichik obraz, model keshlangan) natija qancha yaxshilanadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Operatsion hisobot: SLO, xato byudjeti, burn-rate ogohlantirishlari va Go/No-Go

MAQSAD

30 kunlik daqiqalik so'rov/xato qatori (kunlik to'lqin, 12 % o'sish, ikki insident: keskin va sekin) sintetik yaratiladi. SLO 99,9 % uchun xato byudjeti va uning sarfi, Google SRE Workbook'dagi ko'p oynali burn-rate qoidalari (1 soat/5 daq > 14,4x, 6 soat/30 daq > 6x, 3 kun/6 soat > 1x) hisoblanib, har insidentni aniqlash kechikishi topiladi. Shu qoidalar asosida PrometheusRule (monitoring.coreos.com/v1) YAML generatsiya qilinadi va validatsiya qilinadi. Oxirida kunlik cho'qqi trendi bo'yicha sig'im prognozi va oldingi qadamlar natijalarini jamlovchi Go/No-Go nazorat ro'yxati.

> **Kutilgan natija:** Mavjudlik ~99,92 %, byudjet sarfi ~84 %; keskin insident 5 daqiqada, sekin insident faqat 6 soatlik va 3 kunlik qoidalarda aniqlanadi; PrometheusRule VALID (5 qoida); sig'im ~45 kunga yetadi; QAROR: GO.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Operatsion hisobot: SLO va xato byudjeti, ko'p oynali burn-rate ogohlantirishlari,
PrometheusRule YAML
# generatsiyasi va validatsiyasi, sig'im prognozi va yakuniy Go/No-Go nazorat ro'yxati.
# 30 kunlik daqiqalik so'rov/xato qatori sintetik tarzda yaratiladi (real klasterda —
Prometheus'dan olinadi).
import math
import numpy as np
import pandas as pd
import yaml
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
DAQ = 30 * 24 * 60
t = np.arange(DAQ)
kun = (t % 1440) / 1440
sorov = (120 + 200 * np.sin(np.pi * kun) ** 2) * 60 * (1 + 0.004 * t / 1440)      #
so'rov/daqiqa, 30 kunda +12 % o'sish
sorov = rng.poisson(sorov).astype(float)
xato_ulush = np.full(DAQ, 0.0004)
INSIDENT = [("model fayli buzilgan (keskin)", 9 * 1440 + 600, 40, 0.15),
            ("xotira sizishi (sekin)", 21 * 1440 + 300, 720, 0.012)]
for nom, bosh, dav, e in INSIDENT:
    xato_ulush[bosh:bosh + dav] = e
xato = rng.binomial(sorov.astype(int), xato_ulush).astype(float)

SLO = 0.999
BYUDJET = (1 - SLO) * sorov.sum()
print("=== SLO va xato byudjeti (30 kun) ===")
print(f"So'rovlar: {sorov.sum() / 1e6:.2f} mln, xatolar: {xato.sum():.0f}, mavjudlik: {100 * (1
- xato.sum() / sorov.sum()):.4f} %")
print(f"SLO {100 * SLO:.1f} % -> byudjet {BYUDJET:.0f} xato; sarflangan {100 * xato.sum() /
BYUDJET:.1f} %")
for nom, bosh, dav, e in INSIDENT:
    print(f"  insident '{nom}': {xato[bosh:bosh + dav].sum():.0f} xato = byudjetning {100 *
xato[bosh:bosh + dav].sum() / BYUDJET:.1f} %")
```

```
def burn(oyna):
    """Oyna (daqiqa) bo'yicha burn rate = xato ulushi / (1 - SLO)."""
    cs, cx = np.cumsum(np.r_[0, sorov]), np.cumsum(np.r_[0, xato])
    i = np.arange(1, DAQ + 1); j = np.maximum(0, i - oyna)
    return ((cx[i] - cx[j]) / np.maximum(cs[i] - cs[j], 1)) / (1 - SLO)


# Google SRE Workbook tavsiyasi: (uzun oyna, qisqa oyna, chegara, darajasi)
QOIDA = [("XatoByudjetTez", 60, 5, 14.4, "page"), ("XatoByudjetOrta", 360, 30, 6.0, "page"),
("XatoByudjetSekin", 4320, 360, 1.0, "ticket")]
print("\n=== Ko'p oynali burn-rate ogohlantirishlari ===")
yongan = {}
for nom, uzun, qisqa, ch, dar in QOIDA:
    f = (burn(uzun) > ch) & (burn(qisqa) > ch)
    yongan[nom] = f
    print(f"{nom:17s} ({uzun // 60:2d} soat / {qisqa:3d} daq > {ch:4.1f}x, {dar:6s}):
{int(f.sum()):5d} daqiqa yongan")
print(f"\n{'insident':32s} " + " ".join(f"{n:>17s}" for n, *_ in QOIDA))
for nom, bosh, dav, e in INSIDENT:
    kech = []
    for q, *_ in QOIDA:
        idx = np.flatnonzero(yongan[q][bosh:bosh + dav + 360])
        kech.append(f"{idx[0]:>13d} daq" if len(idx) else f"{'aniqlanmadi':>17s}")
    print(f"{nom:32s} " + " ".join(kech))
print("Tez qoida keskin nosozlikni daqiqalarda ushlaydi; sekin sizishni faqat 6 soat / 3 kun
oynali qoidalar ko'radi.")

# --- PrometheusRule YAML generatsiyasi va validatsiyasi ---
def ifoda(oyna):
    o = f"{oyna // 60}h" if oyna >= 60 else f"{oyna}m"
    return (f'(sum(rate(http_requests_total{{job="tumor-api",code=~"5.."}}[{o}])) / '
            f'sum(rate(http_requests_total{{job="tumor-api"}}[{o}])))')


rule = {"apiVersion": "monitoring.coreos.com/v1", "kind": "PrometheusRule",
        "metadata": {"name": "tumor-api-slo", "namespace": "ai-prod", "labels": {"release":
"kube-prometheus-stack"}},
        "spec": {"groups": [{"name": "tumor-api.slo", "rules": [
            {"alert": nom, "expr": f"{ifoda(uzun)} > ({ch} * {1 - SLO:.3f}) and {ifoda(qisqa)}
> ({ch} * {1 - SLO:.3f})",
             "for": "2m", "labels": {"severity": dar, "service": "tumor-api"},
             "annotations": {"summary": f"Xato byudjeti {ch}x tezlikda yonmoqda",
"runbook_url": "https://wiki.local/runbooks/tumor-api"}}
            for nom, uzun, qisqa, ch, dar in QOIDA] + [
            {"alert": "PodQaytaIshgaTushmoqda", "expr":
'increase(kube_pod_container_status_restarts_total{namespace="ai-prod"}[15m]) > 3',
             "for": "5m", "labels": {"severity": "ticket"}, "annotations": {"summary":
"CrashLoopBackOff ehtimoli"}},
            {"alert": "HPAMaksimumda", "expr":
'kube_horizontalpodautoscaler_status_current_replicas ==
kube_horizontalpodautoscaler_spec_max_replicas',
             "for": "15m", "labels": {"severity": "ticket"}, "annotations": {"summary": "HPA
maxReplicas ga yetdi — sig'im rejasini ko'ring"}}]}]}}
matn = yaml.safe_dump(rule, sort_keys=False, allow_unicode=True, width=200)


def rule_tekshir(r):
    x = []
    if r.get("kind") != "PrometheusRule": x.append("kind noto'g'ri")
    nomlar = []
    for g in r["spec"]["groups"]:
        for a in g["rules"]:
            nomlar.append(a.get("alert"))
```

```
            for k in ("alert", "expr", "for", "labels", "annotations"):
                if k not in a: x.append(f"{a.get('alert')}: '{k}' yo'q")
            if a.get("labels", {}).get("severity") not in ("page", "ticket"): x.append(f"
{a['alert']}: severity noto'g'ri")
            if a["expr"].count("(") != a["expr"].count(")"): x.append(f"{a['alert']}: qavslar
mos emas")
            if not str(a.get("for", "")).rstrip("smh").isdigit(): x.append(f"{a['alert']}:
'for' davomiyligi xato")
            if a.get("labels", {}).get("severity") == "page" and "runbook_url" not in
a.get("annotations", {}):
                x.append(f"{a['alert']}: page ogohlantirishida runbook yo'q")
    if len(nomlar) != len(set(nomlar)): x.append("takroriy alert nomi")
    return x, len(nomlar)


xat, n = rule_tekshir(yaml.safe_load(matn))
print(f"\n=== PrometheusRule: {n} qoida, {len(matn.splitlines())} qator YAML -> {'VALID' if not
xat else xat} ===")
print("\n".join(matn.splitlines()[:16]) + "\n  ...")

# --- Sig'im prognozi: kunlik cho'qqining chiziqli trendi ---
cho = sorov.reshape(30, 1440).max(1) / 60
a, b = np.polyfit(np.arange(30), cho, 1)
SIGIM = 12 * 50 * 0.7                                  # 12 Pod x 50 so'rov/s x 70 % nishon
kun_qoldi = (SIGIM - b) / a - 29 if a > 0 else math.inf
print(f"\n=== Sig'im prognozi ===\nKunlik cho'qqi: {cho[0]:.0f} -> {cho[-1]:.0f} so'rov/s
(trend +{a:.2f}/kun); "
      f"xavfsiz sig'im {SIGIM:.0f} so'rov/s -> ~{kun_qoldi:.0f} kundan keyin yetadi")

# --- Yakuniy Go/No-Go ro'yxati: dastlabki 7 band — oldingi qadamlar natijasi (qo'lda
kiritiladi), qolganlari shu qadamda hisoblanadi ---
ROYXAT = [
    ("Xizmat testlari 11/11 PASS (1-qadam)", True),
    ("Konfiguratsiya validatsiyasi fail-fast (2-qadam)", True),
    ("startupProbe bilan CrashLoop yo'q (2-qadam)", True),
    ("Dockerfile 13/13, compose OK (3-qadam)", True),
    ("K8s manifest 18/18 prod (4-qadam)", True),
    ("Rollout: maxUnavailable=0, canary avtomatik orqaga qaytaradi (5-qadam)", True),
    ("HPA/CA: sakrashda SLO buzilishi <= 5 daq (6-qadam, bulut CA)", True),
    (f"30 kunlik mavjudlik >= {100 * SLO:.1f} %", 1 - xato.sum() / sorov.sum() >= SLO),
    ("Xato byudjeti sarfi < 100 %", xato.sum() < BYUDJET),
    ("Burn-rate qoidalari VALID va ikkala insident aniqlandi", not xat and
all(yongan["XatoByudjetSekin"][b_:b_ + d_ + 360].any() for _, b_, d_, _ in INSIDENT)),
    ("Sig'im kamida 30 kunga yetadi", kun_qoldi >= 30),
]
print("\n=== Go/No-Go nazorat ro'yxati ===")
for nom, ok in ROYXAT:
    print(f"[{'+' if ok else '-'}] {nom}")
print("QAROR:", "GO — ishlab chiqarishga chiqarish mumkin" if all(o for _, o in ROYXAT) else
      "NO-GO — belgilangan bandlar tuzatilmaguncha chiqarilmaydi: " + "; ".join(n for n, o in
ROYXAT if not o))

fig, ax = plt.subplots(2, 1, figsize=(9, 5), sharex=True)
ax[0].plot(t / 1440, sorov / 60, lw=0.4); ax[0].set_ylabel("so'rov/s")
ax[1].plot(t / 1440, 100 - 100 * np.cumsum(xato) / BYUDJET, "r-"); ax[1].axhline(0, color="k",
lw=0.8)
ax[1].set_ylabel("qolgan byudjet, %"); ax[1].set_xlabel("kun"); plt.tight_layout()
print("\n=== Operatsion hisobot tayyor ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== SLO va xato byudjeti`, `=== Ko'p oynali burn-rate ogohlantirishlari ===`, `VALID`, `=== Go/No-Go nazorat ro'yxati ===`, `=== Operatsion hisobot tayyor ===`

TAJRIBA QILIB KO‘RING

- SLO ni 99,95 % ga oshiring — qaror va qaysi bandlar o'zgaradi?
- Kechikish SLO'si (p95 < 200 ms) uchun ham burn-rate qoidasini qo'shing (histogram_quantile ifodasi bilan).
- Insidentlar sonini 4 taga oshirib, byudjet tugaydigan kunni va 'muzlatish' (feature freeze) siyosatini hisobotga yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Xizmatga yangi endpoint va testlar** 1-qadam xizmatiga /v1/explain endpointini qo'shing (v1 logistik regressiya uchun eng katta hissa qo'shgan 5 ta belgi va ularning koeffitsient x qiymati). Kamida 3 ta yangi test yozing (200, 401, 422) va test jadvalini topshiring.	oson	5
2	**Konfiguratsiya sxemasini kengaytirish** 2-qadam sxemasiga 3 ta yangi kalit qo'shing (masalan, redis_url, cache_ttl_s, cors_origins) va har biri uchun tur/diapazon tekshiruvi yozing. To'g'ri va xato konfiguratsiyalar uchun natijalarni hamda manba (standart/ConfigMap/env) jadvalini ko'rsating.	oson	5
3	**Dockerfile qoidalari va obraz hajmi** 3-qadam tahlilchisiga 3 ta qoida qo'shing (.dockerignore borligi, LABEL versiyasi, HEALTHCHECK'da curl o'rniga python yoki wget). Paketlar hajmini taxminiy baholab (python:3.12-slim ~ 130 MB, numpy, scikit-learn, fastapi), bir bosqichli va ko'p bosqichli obraz hajmini jadvalda solishtiring.	oson	5
4	**Probe parametrlarini optimallashtirish** 2-qadam kubelet simulyatorida model yuklanish vaqti 20, 45, 90, 180 s bo'lgan 4 ssenariy uchun startupProbe (period, failureThreshold) ni tanlang: tayyor bo'lish vaqti minimal, restart 0 bo'lsin, osilib qolgan konteyner esa 5 daqiqada qayta ishga tushsin. Natijani jadval va tavsiya ko'rinishida bering.	o'rta	10
5	**Manifest validatorini kengaytirish** 4-qadam tekshiruvchisiga 5 ta yangi qoida qo'shing: NetworkPolicy podSelector Deployment'ga mos, Ingress TLS secretName mavjud (Secret obyekti), HPA metrikasi uchun requests.cpu bor, imagePullPolicy IfNotPresent/Always, PVC readOnly mount. Har qoida uchun uni buzadigan test manifest yozing va natijalar jadvalini topshiring.	o'rta	10
6	**Rollout strategiyasi tanlovi** 5-qadam rolling simulyatorida 10 nusxa, Pod start 60 s, talab 80 % sig'im sharoitida maxSurge (0..3) x maxUnavailable (0..3) setkasini hisoblang: davomiylik, yo'qolgan so'rovlar va qo'shimcha Pod-soniya. Pareto-optimal kombinatsiyalarni ajrating va grafikda ko'rsating.	o'rta	10
7	**Canary statistik kuchi** 5-qadam canary tahliliga ikki ulush uchun z-test qo'shing. v2 xato ulushi 1, 2, 4, 8 % bo'lganda (baza 0,5 %) 5 % bosqichda nuqsonni aniqlash uchun kerakli canary namunalari sonini va umumiy so'rovlar sonini toping; 20 ta urug' bilan takrorlab, aniqlash ehtimolini jadvalda bering.	o'rta	10
8	**HPA va CA sozlash tadqiqoti** 6-qadam simulyatorida o'z yuklama profilingizni (masalan, ertalabki cho'qqi + 2 ta sakrash) bering va kamida 8 konfiguratsiyani (nishon %, min/max, RPS metrikasi, tugun start vaqti, oldindan zaxira, scaleDown oynasi) SLO buzilishi va narx (Pod-	qiyin	15

	soat x narx + tugun-soat x narx) bo'yicha solishtiring. Pareto-chegara grafigini chizing va tanlovni asoslang.		
9	**To'liq joylashtirish to'plami** 1-4 qadamlar asosida bitta skript yozing: u xizmat kodini (main.py), requirements.txt, Dockerfile, docker-compose.yml va dev/prod K8s manifestlarini /tmp/deploy papkasiga generatsiya qilsin, barcha validatorlarni ishga tushirsin va umumiy PASS/FAIL hisobotini chiqarsin. Agar real Docker yoki Kubernetes (minikube, kind, k3s) bo'lsa, docker build va kubectl apply natijasini skrinshot bilan ilova qiling; bo'lmasa simulyatsiya natijalarini bering.	qiyin	15
10	**Operatsion hisobot o'z xizmatingiz uchun** O'z AI xizmatingiz (masalan, tasvir tasnifi, chatbot, tavsiya tizimi) uchun 7-qadamni moslang: SLO'larni (mavjudlik va kechikish) tanlang, 30 kunlik sintetik ma'lumotga 3 ta turli insident kiriting, burn-rate qoidalari va PrometheusRule'ni generatsiya qiling, aniqlash kechikishlarini jadvalda bering, sig'im prognozi va Go/No-Go qarorini raqamlar bilan asoslang (kamida 400 so'z).	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: qandli diabet xavfini baholash API oson

Soha: Tibbiyot

Poliklinikalar tarmog'i bemor laboratoriya ko'rsatkichlari bo'yicha diabet xavfini baholovchi ichki API'ni Kubernetes'ga joylashtiradi. Ish vaqtida 30 so'rov/s, kechasi deyarli nol. Ma'lumot shifoxona tarmog'idan chiqmaydi, model har chorakda yangilanadi.

Model	sklearn diabetes (regressiya -> xavf darajasi), Ridge v1 / GBR v2
Yuklama profili	kunduz 30, tun 2 so'rov/s
SLO	p95 <= 150 ms, 99,5 %
Klaster	3 tugun on-prem
Rollout	RollingUpdate maxSurge 1
HPA	min 2, max 6, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sklearn diabetes (regressiya -> xavf darajasi), Ridge v1 / GBR v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 6, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 1) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

2-variant. Agro: tuproq namligi bo'yicha sug'orish tavsiyasi oson

Soha: Qishloq xo'jaligi

Fermer xo'jaliklari IoT sensorlari har 10 daqiqada ma'lumot yuboradi, xizmat sug'orish kerakmi-yo'qligini qaytaradi. Sensorlar soni mavsumda 2000 dan 15000 gacha o'sadi. Xizmat tuman data-markazidagi kichik k3s klasterida ishlaydi.

Model	make_regression (8 belgi) -> RandomForest
Yuklama profili	2000 -> 15000 qurilma / 10 daq
SLO	p95 <= 500 ms, 99 %
Klaster	k3s, 3 tugun
Rollout	Recreate vs RollingUpdate
HPA	min 1, max 4, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (8 belgi) -> RandomForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 4, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (Recreate vs RollingUpdate) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

3-variant. Transport: taksi narxini dinamik hisoblash oson

Soha: Transport

Taksi agregatori har buyurtmada narxni ML model bilan hisoblaydi; juma kechqurun yuklama 5 barobar oshadi. Kechikish ilova tajribasiga bevosita ta'sir qiladi. Siz canary bilan yangi narx modelini xavfsiz chiqarasiz.

Model	make_regression (12 belgi) -> GradientBoosting
Yuklama profili	80 -> 400 so'rov/s (juma)
SLO	p99 <= 100 ms, 99,9 %
Klaster	bulut, 4 -> 12 tugun
Rollout	canary 5/25/50/100
HPA	min 4, max 30, CPU 65 % + RPS

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (12 belgi) -> GradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 30, CPU 65 % + RPS, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/25/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

4-variant. Telekom: abonent ketishini (churn) bashorat qilish oson

Soha: Telekom

Operator CRM tizimi har abonent qo'ng'irog'ida churn ehtimolini so'raydi; call-markaz ish vaqtida 120 so'rov/s. Model har oy qayta o'qitiladi va A/B sinovdan o'tkaziladi. Konfiguratsiya ConfigMap orqali, API kalit Secret orqali boshqariladi.

Model	make_classification (20 belgi, nomutanosib 9:1) -> LogisticRegression
Yuklama profili	120 so'rov/s, 8:00-20:00
SLO	p95 <= 200 ms, 99,9 %
Klaster	on-prem, 5 tugun
Rollout	blue/green
HPA	min 3, max 10, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (20 belgi, nomutanosib 9:1) -> LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 10, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (blue/green) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

5-variant. Energetika: elektr iste'molini soatlik prognoz qilish oson

Soha: Energetika

Hududiy elektr tarmog'i kompaniyasi har soat 1200 podstansiya bo'yicha keyingi 24 soat iste'molini prognoz qiladi. Yuklama soat boshida qisqa cho'qqi beradi (paketli so'rovlar). Xizmat batch endpointi bilan ishlaydi.

Model	sintetik vaqt qatori (sinus + shovqin) -> Ridge, batch 64
Yuklama profili	soat boshida 1200 so'rov 2 daqiqada
SLO	batch p95 <= 2 s
Klaster	on-prem, 3 tugun
Rollout	RollingUpdate maxUnavailable 0
HPA	min 2, max 8, RPS 20/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik vaqt qatori (sinus + shovqin) -> Ridge, batch 64): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 8, RPS 20/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxUnavailable 0) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

6-variant. Moliya: kredit skoring xizmati oson

Soha: Moliya (bank)

Bank onlayn kredit arizalarini 2 soniyada ko'rib chiqishi kerak; skoring modeli regulyator talabiga ko'ra versiyalanadi va har javob model versiyasi bilan jurnallanadi. Xizmat ikki namespace'da (dev, prod) va NetworkPolicy bilan izolyatsiyalangan.

Model	make_classification (15 belgi) -> LogisticRegression v1 / RandomForest v2
Yuklama profili	25 so'rov/s, cho'qqi 90
SLO	p95 <= 300 ms, 99,95 %
Klaster	on-prem, 3 control-plane + 4 worker
Rollout	canary + qo'lda tasdiq
HPA	min 3, max 8, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (15 belgi) -> LogisticRegression v1 / RandomForest v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 8, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + qo'lda tasdiq) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

7-variant. Ta'lim: insho baholash yordamchisi oson

Soha: Ta'lim

Universitet talabalar inshosini matn belgilari (uzunlik, so'z boyligi, xatolar soni) bo'yicha dastlabki baholovchi xizmatni ishga tushiradi. Sessiya davrida yuklama 10 barobar o'sadi. Xizmat docker-compose bilan sinovdan o'tib, keyin K8s'ga ko'chiriladi.

Model	sintetik matn belgilari (10 belgi) -> Ridge
Yuklama profili	5 -> 50 so'rov/s (sessiya)
SLO	p95 <= 1 s, 99 %
Klaster	docker-compose -> k3s 2 tugun
Rollout	RollingUpdate
HPA	min 1, max 5, CPU 75 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (10 belgi) -> Ridge): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 5, CPU 75 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

8-variant. Sanoat: nasoslarning nosozligini vibratsiya bo'yicha bashorat qilish

o'rta

Soha: Sanoat

Kimyo zavodida 600 nasos sensori har 5 s da vibratsiya spektri xususiyatlarini yuboradi; xizmat nosozlik ehtimolini qaytaradi. Zavod tarmog'i internetdan uzilgan, obrazlar ichki registrdan (NAS) olinadi. Yangilanish faqat ta'mirlash oynasida.

Model	sintetik sensor (16 belgi) -> IsolationForest + LogisticRegression
Yuklama profili	120 so'rov/s doimiy
SLO	p99 <= 250 ms, 99,9 %
Klaster	havo bo'shlig'i (air-gap), 3 tugun
Rollout	ta'mirlash oynasida Rolling
HPA	min 3, max 6, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik sensor (16 belgi) -> IsolationForest + LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 6, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (ta'mirlash oynasida Rolling) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (120 so'rov/s doimiy, havo bo'shlig'i (air-gap), 3 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 250 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

9-variant. Xavfsizlik: kirish tizimida yuz embeddingini solishtirish o'rta

Soha: Xavfsizlik

Korxona turniketlarida xodimning yuz embeddingi (128 o'lchamli) bazadagi 5000 ta bilan solishtiriladi. Smena boshida 15 daqiqada 3000 kirish. Xizmat ishdan chiqsa turniket ochilmaydi, shuning uchun PDB va topologiya tarqalishi majburiy.

Model	sintetik embedding (128) -> kNN (sklearn NearestNeighbors)
Yuklama profili	smena boshida 200 so'rov/s, 15 daq
SLO	p99 <= 300 ms, 99,99 %
Klaster	on-prem, 3 tugun, 2 zona
Rollout	RollingUpdate maxSurge 2
HPA	min 4, max 12, RPS 25/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik embedding (128) -> kNN (sklearn NearestNeighbors)): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 12, RPS 25/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 2) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (smena boshida 200 so'rov/s, 15 daq, on-prem, 3 tugun, 2 zona) — kamida 4 konfiguratsiya, SLO (p99 <= 300 ms, 99,99 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

10-variant. Chakana savdo: mahsulot tavsiya API o'rta

Soha: Chakana savdo

Onlayn do'kon har sahifa ochilganda tavsiya ro'yxatini so'raydi; aksiya kunlari yuklama 8 barobar o'sadi. Natijalar Redis keshida 60 s saqlanadi. Siz compose'da kesh bilan, K8s'da esa HPA va Cluster Autoscaler bilan sinaysiz.

Model	sintetik foydalanuvchi-mahsulot matritsasi -> TruncatedSVD
Yuklama profili	150 -> 1200 so'rov/s (aksiya)
SLO	p95 <= 120 ms, 99,9 %
Klaster	bulut, 3 -> 15 tugun
Rollout	canary 10/50/100
HPA	min 3, max 40, CPU 60 % + RPS

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik foydalanuvchi-mahsulot matritsasi -> TruncatedSVD): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 40, CPU 60 % + RPS, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 10/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (150 -> 1200 so'rov/s (aksiya), bulut, 3 -> 15 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 120 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

11-variant. Logistika: yetkazib berish vaqtini bashorat qilish (ETA) o'rta

Soha: Logistika

Kuryerlik xizmati har buyurtmada ETA ni ML bilan hisoblaydi va mijozga ko'rsatadi; tushlik va kechki cho'qqilar. Model har hafta yangilanadi, xato bashorat mijoz shikoyatiga olib keladi — canary tahlilida MAE ham kuzatiladi.

Model	make_regression (10 belgi, shovqinli) -> HistGradientBoosting
Yuklama profili	40 -> 250 so'rov/s (2 cho'qqi)
SLO	p95 <= 200 ms, 99,9 %
Klaster	bulut, 3 -> 8 tugun
Rollout	canary + MAE tahlili
HPA	min 3, max 15, CPU 65 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (10 belgi, shovqinli) -> HistGradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 15, CPU 65 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + MAE tahlili) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (40 -> 250 so'rov/s (2 cho'qqi), bulut, 3 -> 8 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 200 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

12-variant. Shahar: murojaatlarni avtomatik tasniflash (311 xizmati) o'rta

Soha: Shahar boshqaruvi

Hokimiyat portaliga kelgan fuqarolar murojaatlari (matn belgilari) tegishli bo'limga yo'naltiriladi. Ish kunlari 9:00-11:00 cho'qqi. Xizmat davlat data-markazidagi klasterda, barcha sirlar Secret orqali, tasvirlar root'siz.

Model	sintetik matn TF-IDF (500 belgi) -> LinearSVC
Yuklama profili	10 -> 60 so'rov/s
SLO	p95 <= 400 ms, 99,5 %
Klaster	on-prem, 4 tugun
Rollout	RollingUpdate maxSurge 25 %
HPA	min 2, max 8, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn TF-IDF (500 belgi) -> LinearSVC): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 8, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 25 %) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (10 -> 60 so'rov/s, on-prem, 4 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 400 ms, 99,5 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

13-variant. Ekologiya: suv sifati anomaliyasini aniqlash o'rta

Soha: Ekologiya

Suv ta'minoti korxonasi 80 nazorat nuqtasidan har daqiqada pH, loyqalik, xlor ma'lumotini oladi; anomaliya bo'lsa navbatchiga ogohlantirish. Xizmat kichik edge-klasterda (3 ta mini-PC) ishlaydi, elektr uzilishiga UPS bilan chidamli bo'lishi kerak.

Model	sintetik sensor (6 belgi) -> IsolationForest
Yuklama profili	80 so'rov/daq doimiy
SLO	p95 <= 1 s, 99,9 %
Klaster	edge k3s, 3 mini-PC
Rollout	Recreate taqiqlangan, Rolling
HPA	min 2, max 3, CPU 80 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik sensor (6 belgi) -> IsolationForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 3, CPU 80 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (Recreate taqiqlangan, Rolling) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (80 so'rov/daq doimiy, edge k3s, 3 mini-PC) — kamida 4 konfiguratsiya, SLO (p95 <= 1 s, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

14-variant. Sport: o'yinchi jarohati xavfini baholash o'rta

Soha: Sport

Futbol akademiyasi mashg'ulot yuklamasi (GPS, yurak urishi) bo'yicha o'yinchi jarohati xavfini har kuni hisoblaydi; murabbiylar planshetdan so'raydi. Yuklama kichik, lekin mavjudlik mashg'ulot vaqtida muhim. Xizmat bitta tugunli klasterdan 3 tugunliga ko'chiriladi.

Model	make_classification (12 belgi) -> RandomForest
Yuklama profili	2 -> 15 so'rov/s
SLO	p95 <= 500 ms, 99,5 %
Klaster	1 -> 3 tugun
Rollout	blue/green
HPA	min 1, max 3, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (12 belgi) -> RandomForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 3, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (blue/green) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (2 -> 15 so'rov/s, 1 -> 3 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 500 ms, 99,5 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

15-variant. Media: kontent moderatsiyasi (toksik izohlar) o'rta

Soha: Media

Yangiliklar portali izohlarni chop etishdan oldin toksiklikka tekshiradi; mashhur yangilikda bir necha daqiqada yuklama 20 barobar sakraydi. Xizmat bulutda, Cluster Autoscaler tugun qo'shishi 3 daqiqa oladi — oldindan zaxira siyosatini tanlaysiz.

Model	sintetik matn belgilari (300) -> LogisticRegression
Yuklama profili	20 -> 400 so'rov/s (10 daq)
SLO	p95 <= 250 ms, 99,9 %
Klaster	bulut, 2 -> 12 tugun
Rollout	canary 5/20/100
HPA	min 2, max 40, RPS 30/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (300) -> LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 40, RPS 30/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/20/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (20 -> 400 so'rov/s (10 daq), bulut, 2 -> 12 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 250 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

16-variant. Tibbiyot: tez yordam chaqiruvlarini triaj qilish qiyin

Soha: Tibbiyot (shoshilinch yordam)

103 xizmati operatori chaqiruv belgilarini kiritadi, model ustuvorlik darajasini (1-4) taklif qiladi. Xizmat to'xtashi mumkin emas: ikki klaster (asosiy va zaxira), har yangilanish canary orqali, xato byudjeti juda kichik. Burn-rate ogohlantirishlari pager'ga ulanadi.

Model	make_classification (4 sinf, 18 belgi) -> GradientBoosting
Yuklama profili	15 -> 60 so'rov/s
SLO	p99 <= 300 ms, 99,99 %
Klaster	2 klaster x 3 tugun
Rollout	canary 1/10/50/100
HPA	min 4, max 10, CPU 50 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (4 sinf, 18 belgi) -> GradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 10, CPU 50 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 1/10/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (15 -> 60 so'rov/s, 2 klaster x 3 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 300 ms, 99,99 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

17-variant. Neft-gaz: quduq debitini bashorat qilish qiyin

Soha: Neft-gaz

Kompaniya 900 quduq bo'yicha har soat debit prognozini hisoblaydi va dispetcherga ko'rsatadi. Klaster dala ofisida, internet sun'iy yo'ldosh orqali sekin, shuning uchun obrazlar kichik bo'lishi va ichki registrda keshlanishi shart. Dockerfile hajmini optimallashtirasiz.

Model	sintetik vaqt qatori (14 belgi) -> HistGradientBoosting
Yuklama profili	900 so'rov / soat boshida
SLO	batch p95 <= 3 s, 99,5 %
Klaster	dala ofisi, 3 tugun, sekin WAN
Rollout	RollingUpdate, obraz oldindan tortiladi
HPA	min 2, max 4, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik vaqt qatori (14 belgi) -> HistGradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 4, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate, obraz oldindan tortiladi) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (900 so'rov / soat boshida, dala ofisi, 3 tugun, sekin WAN) — kamida 4 konfiguratsiya, SLO (batch p95 <= 3 s, 99,5 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

18-variant. Sug'urta: avtomobil zarari da'vosini firibgarlikka tekshirish qiyin

Soha: Sug'urta

Sug'urta kompaniyasi har da'voni firibgarlik ehtimoli bo'yicha baholaydi; da'volar ish kunlari tekis keladi, oy oxirida 3 barobar. Regulyator har model versiyasining audit izini talab qiladi: x-model-version, sha256 va konfiguratsiya xeshi jurnallanadi.

Model	make_classification (25 belgi, 19:1) -> RandomForest v1 / GBC v2
Yuklama profili	10 -> 30 so'rov/s (oy oxiri)
SLO	p95 <= 800 ms, 99,9 %
Klaster	on-prem, 3 control-plane + 3 worker
Rollout	canary + audit
HPA	min 2, max 6, CPU 65 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (25 belgi, 19:1) -> RandomForest v1 / GBC v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 6, CPU 65 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + audit) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (10 -> 30 so'rov/s (oy oxiri), on-prem, 3 control-plane + 3 worker) — kamida 4 konfiguratsiya, SLO (p95 <= 800 ms, 99,9 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

19-variant. Davlat xizmatlari: hujjat turini tasniflash (OCR natijasi) qiyin

Soha: Davlat xizmatlari

Yagona interaktiv portalda yuklangan skanlar OCR'dan keyin matn belgilari bo'yicha hujjat turiga (pasport, diplom, ma'lumotnoma ...) ajratiladi. Murojaatlar soni yil davomida o'sib bormoqda — sig'im prognozi va HPA maxReplicas rejasini tuzasiz.

Model	sintetik matn belgilari (200) -> MultinomialNB / LogisticRegression
Yuklama profili	50 so'rov/s, oyiga +5 %
SLO	p95 <= 600 ms, 99,9 %
Klaster	davlat DC, 6 tugun
Rollout	RollingUpdate + canary
HPA	min 3, max 20, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (200) -> MultinomialNB / LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 20, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate + canary) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (50 so'rov/s, oyiga +5 %, davlat DC, 6 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 600 ms, 99,9 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

20-variant. Kiberxavfsizlik: login urinishlarida anomaliyani aniqlash qiyin

Soha: Kiberxavfsizlik

Korporativ SSO tizimi har login urinishini (IP, vaqt, qurilma belgilari) xavf bali bo'yicha baholaydi; hujum paytida yuklama 50 barobar sakraydi va aynan shu payt xizmat ishlashi shart. Rate-limit, NetworkPolicy va tez masshtablash muhim.

Model	sintetik log belgilari (10) -> IsolationForest + LogisticRegression
Yuklama profili	50 -> 2500 so'rov/s (hujum)
SLO	p99 <= 150 ms, 99,95 %
Klaster	bulut, 4 -> 20 tugun
Rollout	canary 5/50/100
HPA	min 4, max 60, RPS 40/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik log belgilari (10) -> IsolationForest + LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 60, RPS 40/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (50 -> 2500 so'rov/s (hujum), bulut, 4 -> 20 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 150 ms, 99,95 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20